JS

GUÍA INTERACTIVA

Node.js y Express

JavaScript en el servidor: el event loop, asincronía, streams y APIs REST con Express 5, Zod, JWT y tests. Con ejemplos que se ejecutan de verdad.

Versión PDF · 30 de octubre de 2026
16 capítulos · Los ejemplos interactivos están en la app (npm run dev)

Índice

Empezá acá

1. Cómo usar esta guía

Parte 1 · Node.js

2. Módulos, npm y TypeScript
3. El event loop
4. Callbacks, promesas y async/await
5. Archivos, rutas y streams
6. Eventos y el proceso
7. HTTP sin frameworks

Parte 2 · Express

8. Express: rutas y respuestas
9. Middlewares
10. Diseñar una API REST
11. Validar con Zod
12. Capas y base de datos
13. Autenticación y seguridad
14. Tests con node:test
15. A producción

Parte 3 · Repaso

16. Chuleta y preguntas de entrevista

Empezá acá

Qué es Node, cómo se generan las salidas de los ejemplos y cómo está armada la guía.

Capítulo 1

Cómo usar esta guía

Node en un párrafo, Node vs el navegador, y ejemplos que se ejecutan de verdad.

Esta guía cubre **Node.js** (JavaScript fuera del navegador) y **Express** (el framework más usado para hacer APIs con Node). Es el backend "liviano" de un perfil fullstack: lo que te van a preguntar si el puesto dice *React + Node*.

Qué es Node

Node es el motor de JavaScript de Chrome (**V8**) más una librería en C (**libuv**) que le da acceso al sistema: archivos, red, procesos, timers. Con eso, el mismo lenguaje del front sirve para escribir servidores, scripts y herramientas (Vite, ESLint y TypeScript son programas de Node).

Lo que lo hace distinto de otros backends: corre tu código en **un solo hilo** y nunca se queda esperando una lectura de disco o una respuesta de la red. Mientras espera, atiende otra cosa. Eso es el **event loop**, el tema más preguntado de Node en entrevistas, y tiene su capítulo.

	En el navegador	En Node
Globales	`window`, `document`	`process`, `globalThis`

	En el navegador	En Node
Acceso a	El DOM, `localStorage`, la cámara…	Archivos, red, procesos, variables de entorno
Módulos	`<script type="module">`	`import` / `export` (y el viejo `require`)
Igual en los dos	`fetch`, `URL`, `setTimeout`, `AbortController`, `crypto.randomUUID()`, `structuredClone`, `TextEncoder`	

Los ejemplos se ejecutan de verdad

Cada ejemplo de la guía es un archivo de `src/demos/node/`, y debajo está **lo que imprimió al ejecutarlo**:

`src/demos/node/01-node/event-loop/orden.ts`

```
import { readFile } from 'node:fs'

console.log('1. Código sincrónico: corre de un tirón, primero que
  todo')

setTimeout(() => console.log('6. setTimeout(0): fase de timers'), 0)
Promise.resolve().then(() => console.log('3. Promesa resuelta:
  microtarea'))
queueMicrotask(() => console.log('4. queueMicrotask: otra
  microtarea'))
process.nextTick(() => console.log('5. process.nextTick'))

readFile(import.meta.filename, () => {
  console.log('7. Terminó de leer el archivo: callback de I/O')
  // Adentro de un callback de I/O, setImmediate siempre gana a
    setTimeout(0)
  setTimeout(() => console.log('9. setTimeout(0) desde el I/O'), 0)
  setImmediate(() => console.log('8. setImmediate desde el I/O: fase
    check'))
})

console.log('2. Más código sincrónico')
```

$ node orden.ts · Node 24.22.1

```
1. Código sincrónico: corre de un tirón, primero que todo
2. Más código sincrónico
3. Promesa resuelta: microtarea
4. queueMicrotask: otra microtarea
5. process.nextTick
6. setTimeout(0): fase de timers
7. Terminó de leer el archivo: callback de I/O
8. setImmediate desde el I/O: fase check
9. setTimeout(0) desde el I/O
```

Esas salidas no están escritas a mano: `npm run salidas` corre cada ejemplo con Node y guarda lo que imprime. Para las apps de Express, levanta la app, le hace pedidos HTTP de verdad y guarda las respuestas:

`src/demos/node/02-express/hola.ts`

```ts
import express from 'express'

export const app = express()

// Una ruta = método HTTP + camino + función que responde
app.get('/', (_req, res) => {
  res.send('Hola desde Express')
})

app.get('/salud', (_req, res) => {
  res.json({ ok: true, version: '1.0.0' })
})

// Solo si se ejecuta este archivo (node hola.ts); los tests lo
  importan sin levantarlo
if (import.meta.main) {
  app.listen(3000, () => console.log('Escuchando en
    http://localhost:3000'))
}
```

PEDIDOS A LA APP · Node 24.22.1 · Express 5.3.0

GET `/`

← 200 OK

```
Hola desde Express
```

GET `/salud`

← 200 OK

```
{
  "ok": true,
  "version": "1.0.0"
}
```

GET `/no-existe`

← 404 Not Found

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>Cannot GET /no-existe</pre>
</body>
</html>
```

Podés correr cualquiera vos: `node src/demos/node/02-express/hola.ts` y abrí `http://localhost:3000`.

TypeScript sin compilar

Los ejemplos son `.ts` y se ejecutan con `node archivo.ts`, **sin compilar**: desde Node 22.18 (y en Node 24, la LTS que tenés instalada), Node borra los tipos al cargar el archivo y ejecuta lo que queda. Lo vemos en módulos. Para chequear los tipos se sigue usando `tsc`; en este proyecto, `npm run build` chequea también estos ejemplos.

> ℹ️ **Versiones**
>
> Escrita en octubre de 2026. Las salidas se generaron con **Node 24** (LTS hasta abril de 2028) y **Express 5**. Node 26 pasó a LTS el 28 de octubre de 2026: todo lo de la guía funciona igual. Si mantenés un proyecto con Express 4, el repaso tiene las diferencias.

Cómo está armada

1. **Node.js**: módulos y npm, el event loop, asincronía, archivos y streams, el proceso y sus eventos, y un servidor HTTP sin frameworks (para entender qué resuelve Express).
2. **Express**: rutas, middlewares, cómo diseñar una API REST, validación con Zod, una arquitectura en capas con MySQL, autenticación, tests y lo necesario para producción.
3. **Repaso**: chuleta, diferencias entre Express 4 y 5, y preguntas de entrevista.

Para la base de datos, esta guía usa lo que viste en la guía de MySQL.

Parte 1 · Node.js

El runtime: módulos, el event loop, asincronía, archivos, el proceso y HTTP sin frameworks.

Capítulo 2

Módulos, npm y TypeScript

ES modules y CommonJS, package.json, semver y ejecutar .ts sin compilar.

Un proyecto de Node es una carpeta con un `package.json` y archivos que se importan entre sí. Este capítulo cubre los dos sistemas de módulos, npm y cómo ejecutar TypeScript directo.

Módulos ES (import / export)

Es el estándar de JavaScript, el mismo que usás en React. Cada archivo es un módulo: lo que no exporta queda privado.

src/demos/node/01-node/modulos/main.ts

```ts
import { readFile } from 'node:fs/promises' // los módulos de Node,
  con el prefijo node:
import { basename } from 'node:path'
import rango, { conIva, formatear, type Moneda } from './precios.ts'
  // con la extensión

const moneda: Moneda = 'PYG'
console.log(rango(3000, 4000))
console.log(formatear(conIva(1000)))
console.log(formatear(18_000_000, moneda))

// Dónde está este archivo (en CommonJS eran __dirname y __filename)
console.log('Carpeta:', basename(import.meta.dirname))
console.log('Archivo:', basename(import.meta.filename))
```

```
// await afuera de una función: se puede en el nivel superior de un
  módulo ES
const paquete = JSON.parse(await readFile(new
  URL('../../../../../package.json', import.meta.url), 'utf8'))
console.log('Proyecto:', paquete.name, '· type:', paquete.type)
```

src/demos/node/01-node/modulos/precios.ts

```
// Un módulo ES: lo que no se exporta queda privado a este archivo.
const IVA = 0.1

export type Moneda = 'USD' | 'PYG'

export function conIva(monto: number) {
  return Math.round(monto * (1 + IVA))
}

export function formatear(monto: number, moneda: Moneda = 'USD') {
  return new Intl.NumberFormat('es-PY', { style: 'currency', currency:
    moneda }).format(monto)
}

// Un export por defecto (como mucho uno por archivo)
export default function rango(min: number, max: number) {
  return `${formatear(min)} – ${formatear(max)}`
}
```

$ node main.ts · Node 24.22.1

```
USD 3.000,00 – USD 4.000,00
USD 1.100,00
Gs. 18.000.000
Carpeta: modulos
Archivo: main.ts
Proyecto: guia-react · type: module
```

- Los módulos propios de Node se importan con el prefijo `node:` (`node:fs/promises`, `node:path`). Funciona sin él, pero así queda claro que no es un paquete de npm.
- En Node, los imports relativos llevan **la extensión** (`./precios.ts`). En un proyecto con Vite o Next no hace falta porque el bundler la completa; Node no adivina.

- `import.meta.dirname` e `import.meta.filename` son la carpeta y el archivo actuales.
- **Top-level await**: en un módulo ES se puede usar `await` fuera de una función.

CommonJS (require)

El sistema original de Node, que vas a encontrar en código viejo, en muchas librerías y en los archivos de configuración `.cjs`:

× CommonJS

```
// precios.js
const IVA = 0.1
function conIva(monto) {
  return monto * (1 + IVA)
}
module.exports = { conIva }

// main.js
const { conIva } = require('./precios')
const path = require('node:path')
console.log(__dirname)
```

✓ Módulos ES

```
// precios.ts
const IVA = 0.1
export function conIva(monto: number) {
  return monto * (1 + IVA)
}

// main.ts
import { conIva } from './precios.ts'
import path from 'node:path'
console.log(import.meta.dirname)
```

	CommonJS	Módulos ES
Sintaxis	`require()` / `module.exports`	`import` / `export`

	CommonJS	Módulos ES
Cuándo carga	Al ejecutar el `require` (sincrónico)	Antes de ejecutar el archivo (se analizan los imports primero)
`await` en el nivel superior	No	Sí
`__dirname`, `__filename`	Sí	`import.meta.dirname`, `import.meta.filename`
Extensión en imports relativos	Opcional	Obligatoria

¿Cuál usa un archivo? `.mjs` / `.mts` siempre es ES y `.cjs` / `.cts` siempre CommonJS. Un `.js` o `.ts` depende del `"type"` del `package.json` más cercano: `"module"` es ES; sin `"type"`, CommonJS. Para un proyecto nuevo: `"type": "module"`. Desde Node 22.12, un módulo CommonJS puede hacer `require()` de un módulo ES (salvo que use top-level await), así que mezclar ya no es el dolor de cabeza que era.

package.json y npm

```
{
  "name": "api-busqueda",
  "type": "module",
  "scripts": {
    "dev": "node --watch --env-file=.env src/servidor.ts",
    "start": "node src/servidor.ts",
    "test": "node --test"
  },
  "dependencies": {
    "express": "^5.3.0",
    "zod": "^4.6.7"
  },
  "devDependencies": {
    "@types/express": "^5.0.6",
    "typescript": "^7.0.2"
  },
  "engines": { "node": ">=24" }
}
```

- `dependencies`: lo que la app necesita para correr. `devDependencies`: solo para desarrollar (tipos, tests, linters). En producción se instala con `npm ci --omit=dev`.
- **Scripts**: `npm run dev`, `npm test`, `npm start`. Adentro de un script, los binarios de `node_modules/.bin` están en el `PATH`.
- `node --watch` reinicia el proceso cuando cambia un archivo (antes se usaba `nodemon`), y `--env-file` carga un `.env` (antes, `dotenv`).

Comando	Qué hace
`npm install express`	Agrega una dependencia (y la anota en `package.json`)
`npm install -D @types/express`	Agrega una dependencia de desarrollo
`npm install`	Instala todo lo del `package.json`, respetando el lock
`npm ci`	Instalación limpia y exacta desde `package-lock.json` (para CI y deploys)
`npm outdated` / `npm update`	Ver y actualizar dentro de los rangos permitidos
`npx paquete`	Ejecuta el binario de un paquete sin instalarlo globalmente
`npm audit`	Busca vulnerabilidades conocidas en las dependencias

Versiones (semver): `5.3.0` es *mayor.menor.parche*. Un cambio de mayor puede romper cosas. `^5.3.0` acepta cualquier `5.x.x` desde la 5.3.0; `~5.3.0`, solo parches `5.3.x`. El `package-lock.json` anota la versión exacta de todo el árbol de dependencias: se sube al repositorio para que todos instalen lo mismo.

TypeScript con `node archivo.ts`

Node 24 ejecuta `.ts` directamente: **borra los tipos** (*type stripping*) y corre el JavaScript que queda. No chequea tipos ni compila. Por eso no acepta lo que en TypeScript **genera código**, como un `enum`:

src/demos/node/01-node/modulos/enum.ts

```ts
// Node ejecuta TypeScript borrando los tipos, sin compilar.
// Lo que no se puede borrar (porque genera código), como un enum, no
  anda.
// @ts-expect-error: erasableSyntaxOnly marca lo mismo que Node
enum Seniority {
  Junior,
  Senior,
}

console.log(Seniority.Senior)
```

$ node enum.ts · Node 24.22.1

```
node:internal/modules/run_main:107
    triggerUncaughtException(
    ^

file://~/guia-react/src/demos/node/01-node/modulos/enum.ts:4
    // @ts-expect-error: erasableSyntaxOnly marca lo mismo que Node
  > enum Seniority {
      Junior,
      Senior,
  > }

SyntaxError [ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX]: TypeScript enum is
not supported in strip-only mode {
  code: 'ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX'
}

Node.js v24.22.1
```

El proceso terminó con código 1

Lo que no se puede borrar: `enum`, `namespace` con código, los *parameter properties* de las clases (`constructor(private x: number)`) e `import x = require()`. En lugar de un `enum`, una unión de strings (`type Seniority = 'junior' | 'senior'`) o un objeto `as const`.

En el `tsconfig.json`, la opción `"erasableSyntaxOnly": true` hace que TypeScript marque esas mismas cosas en el editor, y `"allowImportingTsExtensions": true` permite escribir `./precios.ts`. Para chequear los tipos: `tsc --noEmit` (en CI o como script `npm run typecheck`).

💡 ¿Y tsx, ts-node o compilar?

Antes se usaba **tsx** o **ts-node** para correr TypeScript, o se compilaba con `tsc` a una carpeta `dist/` y se ejecutaba el JavaScript. Siguen siendo válidos (y compilar es lo habitual si publicás una librería), pero para una API, `node archivo.ts` alcanza.

🎤 En una entrevista

"¿Diferencia entre CommonJS y ES modules?" → "CommonJS es el sistema original de Node: require y module.exports, carga sincrónica en el momento del require. ES modules es el estándar de JavaScript: import y export, se analizan antes de ejecutar (por eso permiten tree shaking y top-level await). En Node se elige por la extensión (.mjs/.cjs) o por el campo type del package.json. Hoy un proyecto nuevo usa ES modules."

Capítulo 3

El event loop

Un hilo, microtareas y fases, en qué orden corre cada cosa, no bloquear y worker threads.

Node ejecuta tu JavaScript en **un solo hilo**. Aun así, un servidor de Node atiende miles de conexiones a la vez. El truco es el **event loop**: Node nunca espera sentado una operación lenta (disco, red, un timer); la delega, sigue con otra cosa, y cuando la operación termina, ejecuta tu callback.

La idea

1. Tu código corre en el **call stack**, de un tirón, hasta que termina.
2. Lo lento (leer un archivo, una consulta a MySQL, un `fetch`) lo hace **libuv** por fuera de tu JavaScript: con el sistema operativo o con un grupo de hilos propio.
3. Cuando una de esas operaciones termina, su callback queda **en una cola**.
4. El event loop toma callbacks de las colas, de a uno, **cuando el call stack está vacío**.

Un mozo que toma el pedido de una mesa, lo lleva a la cocina y atiende otra mesa mientras cocinan: no se queda parado al lado del horno.

En qué orden corre cada cosa

`src/demos/node/01-node/event-loop/orden.ts`

```
import { readFile } from 'node:fs'
```

```
console.log('1. Código sincrónico: corre de un tirón, primero que
  todo')

setTimeout(() => console.log('6. setTimeout(0): fase de timers'), 0)
Promise.resolve().then(() => console.log('3. Promesa resuelta:
  microtarea'))
queueMicrotask(() => console.log('4. queueMicrotask: otra
  microtarea'))
process.nextTick(() => console.log('5. process.nextTick'))

readFile(import.meta.filename, () => {
  console.log('7. Terminó de leer el archivo: callback de I/O')
  // Adentro de un callback de I/O, setImmediate siempre gana a
    setTimeout(0)
  setTimeout(() => console.log('9. setTimeout(0) desde el I/O'), 0)
  setImmediate(() => console.log('8. setImmediate desde el I/O: fase
    check'))
})

console.log('2. Más código sincrónico')
```

$ node orden.ts · Node 24.22.1

```
1. Código sincrónico: corre de un tirón, primero que todo
2. Más código sincrónico
3. Promesa resuelta: microtarea
4. queueMicrotask: otra microtarea
5. process.nextTick
6. setTimeout(0): fase de timers
7. Terminó de leer el archivo: callback de I/O
8. setImmediate desde el I/O: fase check
9. setTimeout(0) desde el I/O
```

1. **El código sincrónico** corre primero, entero. Ningún callback interrumpe una función que se está ejecutando.
2. Después, las **microtareas**: los `.then` de promesas resueltas (y lo que sigue a un `await`), `queueMicrotask` y `process.nextTick`. Se vacían **todas** antes de seguir.
3. Después, el loop recorre sus **fases**, cada una con su cola:

Fase	Qué ejecuta
timers	Los `setTimeout` y `setInterval` cuyo tiempo ya pasó
poll	Los callbacks de I/O terminado (archivos, red) y espera nuevos

Fase	Qué ejecuta
check	Los `setImmediate`
close	Los eventos `close` (un socket que se cerró)

Entre callback y callback, el loop vuelve a vaciar las microtareas. Por eso una promesa resuelta siempre se adelanta a cualquier timer.

> ⚠️ **Los detalles que caen en una entrevista**
>
> - `setTimeout(fn, 0)` no es "ahora": es "en la próxima fase de timers, como pronto".
> - `setTimeout(0)` **vs** `setImmediate` en el código principal: el orden **no está garantizado** (depende de cuánto tardó en arrancar el proceso). Adentro de un callback de I/O, `setImmediate` siempre va primero, como en el ejemplo.
> - `process.nextTick` es una cola propia de Node que corre antes que las promesas… en CommonJS. En un módulo ES (como este ejemplo), el archivo se ejecuta adentro de una microtarea, así que las promesas que creó se vacían antes. Es un detalle fino: lo importante es que las dos corren antes que cualquier timer o I/O.

No bloquees el event loop

Si un callback tarda mucho **en tu código** (un cálculo pesado, un `JSON.parse` de 200 MB, un `readFileSync` de un archivo enorme), no hay nadie más para atender lo demás: los timers se atrasan y el servidor deja de responder a **todos** los usuarios.

src/demos/node/01-node/event-loop/bloquear.ts

```ts
const inicio = performance.now()
const redondear = (ms: number) => Math.round(ms / 100) * 100

setTimeout(() => {
  console.log(`El timer de 100 ms se ejecutó a los
    ~${redondear(performance.now() - inicio)} ms`)
```

```
}, 100)

// Medio segundo de cálculo sincrónico: mientras tanto, el event loop
  no puede hacer NADA
// (ni timers, ni responder pedidos HTTP, ni leer archivos).
let vueltas = 0
while (performance.now() - inicio < 500) vueltas++
console.log(`Bucle sincrónico terminado
  (~${redondear(performance.now() - inicio)} ms)`)
```

$ node bloquear.ts · Node 24.22.1

```
Bucle sincrónico terminado (~500 ms)
El timer de 100 ms se ejecutó a los ~500 ms
```

El timer pedía 100 ms y corrió a los 500: el loop estaba ocupado con el `while`. En un servidor, eso son 500 ms en los que ningún pedido avanza.

Lo que bloquea, y qué hacer:

- **Las versiones `Sync`** (`readFileSync`, `execSync`, `scryptSync`…): bien en un script o al arrancar la app; nunca adentro de un pedido HTTP.
- **Cálculos pesados** (procesar imágenes, generar PDFs, cifrado): a un **worker thread**, a otro proceso o a una cola de trabajos.
- **Expresiones regulares** que explotan con ciertos textos (*ReDoS*) y bucles sobre listas enormes: partir el trabajo o mover el procesamiento.

Worker threads

Un **worker** es otro hilo con su propio event loop y su propia memoria. Se comunican con mensajes:

`src/demos/node/01-node/event-loop/worker.ts`

```
import { isMainThread, parentPort, Worker, workerData } from
  'node:worker_threads'
import { fibonacci } from './fibonacci.ts'
```

```ts
if (isMainThread) {
  // Hilo principal: manda el cálculo a otro hilo y sigue atendiendo lo suyo
  const worker = new Worker(import.meta.filename, { workerData: 35 })
  worker.on('message', (resultado) => {
    console.log(`Worker: fibonacci(35) = ${resultado}`)
    clearInterval(latido)
  })

  let latidos = 0
  const latido = setInterval(() => {
    latidos++
    if (latidos === 1) console.log('Principal: sigo libre mientras el worker calcula')
  }, 10)
} else {
  // Este mismo archivo, ejecutado en el worker
  parentPort?.postMessage(fibonacci(workerData))
}
```

src/demos/node/01-node/event-loop/fibonacci.ts

```ts
// Un cálculo pesado a propósito (recursivo, sin memoria)
export function fibonacci(n: number): number {
  return n < 2 ? n : fibonacci(n - 1) + fibonacci(n - 2)
}
```

$ node worker.ts · Node 24.22.1

```
Principal: sigo libre mientras el worker calcula
Worker: fibonacci(35) = 9227465
```

El hilo principal siguió con su `setInterval` mientras el worker calculaba. Crear un worker cuesta (unos milisegundos y memoria): para muchas tareas chicas se usa un **pool** de workers (la librería **Piscina**).

ℹ️ ¿Y para usar todos los núcleos?

Un proceso de Node usa un núcleo para tu JavaScript. Para aprovechar una máquina con 8 núcleos se levantan **varios procesos** de la app: con el módulo `node:cluster`, con PM2, o (lo más común hoy) con varias réplicas del contenedor detrás de un balanceador.

🎤 En una entrevista

"¿Cómo hace Node para atender muchos pedidos con un solo hilo?" → "El JavaScript corre en un hilo, pero el I/O no lo hace ese hilo: Node lo delega al sistema operativo o al thread pool de libuv y sigue atendiendo otros pedidos. Cuando el I/O termina, el callback entra en una cola y el event loop lo ejecuta cuando el call stack está vacío. Por eso Node es muy bueno para aplicaciones con mucho I/O (APIs, tiempo real) y malo para trabajo pesado de CPU, que bloquea el loop: eso va a worker threads o a otro servicio."

Capítulo 4

Callbacks, promesas y async/await

Los tres estilos, en serie o en paralelo, Promise.all y compañía, cancelar y timeouts.

Casi todo en Node es asincrónico. La forma de escribirlo pasó por tres etapas, y en un trabajo real vas a encontrar las tres.

Callbacks, promesas y async/await

src/demos/node/01-node/async/estilos.ts

```
import { readFile } from 'node:fs'
import { readFile as readFileAsync } from 'node:fs/promises'
import { promisify } from 'node:util'

const archivo = new URL('./ofertas.json', import.meta.url)

// 1) La API de promesas (node:fs/promises) con async/await: lo que se
  usa hoy
async function contarOfertas() {
  try {
    const texto = await readFileAsync(archivo, 'utf8')
    return JSON.parse(texto).length
  } catch (error) {
    console.error('No se pudo leer:', (error as Error).message)
    return 0
  }
}
console.log('1. async/await:', await contarOfertas(), 'ofertas')

// Si falla, el await lanza el error y lo agarra el catch
try {
  await readFileAsync(new URL('./no-existe.json', import.meta.url),
    'utf8')
```

```
} catch (error) {
  console.log('   Un archivo que no existe:', (error as
    NodeJS.ErrnoException).code)
}

// 2) Una función vieja de callbacks convertida en promesa
const leer = promisify(readFile)
console.log('2. promisify:', JSON.parse(await leer(archivo,
  'utf8')).length, 'ofertas')

// 3) Callback "error primero": el estilo original de Node, que vas a
  ver en código viejo
readFile(archivo, 'utf8', (error, texto) => {
  if (error) return console.error('No se pudo leer:', error.message)
  console.log('3. Callback:', JSON.parse(texto).length, 'ofertas')
})
console.log('   (el callback todavía no corrió: readFile no bloquea)')
```

$ node estilos.ts · Node 24.22.1

```
1. async/await: 3 ofertas
   Un archivo que no existe: ENOENT
2. promisify: 3 ofertas
   (el callback todavía no corrió: readFile no bloquea)
3. Callback: 3 ofertas
```

1. **async/await** con las APIs de promesas (`node:fs/promises`, `fetch`, los drivers de bases de datos): lo que se escribe hoy. Los errores se manejan con `try/catch`, como en código sincrónico.
2. `promisify` convierte una función vieja de callbacks en una que devuelve una promesa.
3. **Callbacks "error primero"**: la función recibe `(error, resultado)`. Si hay error, el primer argumento lo trae. Es el estilo original de Node; el problema es que anidarlos lleva al *callback hell*:

```
leerUsuario(id, (err, usuario) => {
  if (err) return manejar(err)
  buscarPostulaciones(usuario, (err, postulaciones) => {
    if (err) return manejar(err)
    guardarReporte(postulaciones, (err) => {
      if (err) return manejar(err)
      console.log('listo')
```

```
    })
  })
})
```

Fijate en la última línea del ejemplo: el callback de `readFile` corre **después** del `console.log` que está debajo. `readFile` no espera: registra el callback y sigue.

En serie o en paralelo

Un error común: `await` uno por uno cosas que **no dependen** entre sí.

src/demos/node/01-node/async/paralelo.ts

```
import { setTimeout as esperar } from 'node:timers/promises'

// Simula pedidos a APIs que tardan distinto
async function pedir(nombre: string, ms: number, falla = false) {
  await esperar(ms)
  if (falla) throw new Error(`${nombre} respondió 500`)
  return `${nombre} (${ms} ms)`
}

const reloj = () => {
  const inicio = performance.now()
  return () => `~${Math.round((performance.now() - inicio) / 100) *
    100} ms`
}

// Uno después del otro: los tiempos se suman
let tiempo = reloj()
await pedir('empresas', 300)
await pedir('ofertas', 200)
await pedir('postulaciones', 100)
console.log('En serie:', tiempo())

// Todos a la vez: tarda lo que tarde el más lento
tiempo = reloj()
const resultados = await Promise.all([pedir('empresas', 300),
  pedir('ofertas', 200), pedir('postulaciones', 100)])
console.log('Promise.all:', tiempo(), resultados)

// Promise.all falla apenas falla uno
```

```ts
try {
  await Promise.all([pedir('empresas', 300), pedir('ofertas', 100,
    true)])
} catch (error) {
  console.log('Promise.all con un error:', (error as Error).message)
}

// allSettled espera a todos y te dice cuál salió bien y cuál no
const todos = await Promise.allSettled([pedir('empresas', 300),
  pedir('ofertas', 100, true)])
console.log('allSettled:', todos.map((r) => (r.status === 'fulfilled'
  ? r.value : `✗ ${r.reason.message}`)))

// race: el primero que termine (bien o mal). any: el primero que
  termine bien
console.log('race:', await Promise.race([pedir('lento', 300),
  pedir('rápido', 100)]))
console.log('any:', await Promise.any([pedir('falla', 50, true),
  pedir('anda', 150)]))
```

$ node paralelo.ts · Node 24.22.1

```
En serie: ~600 ms
Promise.all: ~300 ms [ 'empresas (300 ms)', 'ofertas (200 ms)',
'postulaciones (100 ms)' ]
Promise.all con un error: ofertas respondió 500
allSettled: [ 'empresas (300 ms)', '✗ ofertas respondió 500' ]
race: rápido (100 ms)
any: anda (150 ms)
```

Función	Termina cuando...	Si una falla...
`Promise.all`	Terminan **todas**	Falla apenas falla una (las demás siguen corriendo, pero se ignoran)
`Promise.allSettled`	Terminan todas	No falla nunca: te da el estado de cada una
`Promise.race`	Termina **la primera**, bien o mal	Falla si la primera en terminar falla
`Promise.any`	Termina la primera **que sale bien**	Falla solo si fallan todas

> 💡 **Cuántas a la vez**
>
> `Promise.all` con 10.000 pedidos a una API los dispara **todos juntos** y probablemente te bloqueen o se caiga algo. Para muchos elementos se procesan de a lotes (por ejemplo, de a 10) o con una librería como **p-limit**.

Cancelar y poner un límite de tiempo

Una operación sin timeout puede quedarse colgada para siempre (un servidor que no responde). `AbortSignal` es el mecanismo estándar para cancelar: lo aceptan `fetch`, los timers de `node:timers/promises`, `fs`, los streams y cada vez más librerías.

src/demos/node/01-node/async/cancelar.ts

```ts
import { setTimeout as esperar } from 'node:timers/promises'

// Una operación lenta que acepta una señal para cancelarla (como
  fetch)
async function buscarOfertas(signal: AbortSignal) {
  await esperar(1000, undefined, { signal })
  return ['Frontend React Senior']
}

// AbortSignal.timeout: se cancela sola si tarda más de 200 ms
try {
  await buscarOfertas(AbortSignal.timeout(200))
} catch (error) {
  console.log('Con timeout:', (error as Error).name)
}

// AbortController: la cancelás vos (por ejemplo, el usuario cerró la
  página)
const controller = new AbortController()
setTimeout(() => controller.abort(), 100)
try {
  await buscarOfertas(controller.signal)
} catch (error) {
  console.log('Cancelada a mano:', (error as Error).name)
}
```

$ node cancelar.ts · Node 24.22.1

```
Con timeout: AbortError
Cancelada a mano: AbortError
```

- `AbortSignal.timeout(ms)`: una señal que se cancela sola.
- `new AbortController()`: la cancelás vos con `controller.abort()`. En el front es lo que usás para cancelar un `fetch` cuando el componente se desmonta.
- `AbortSignal.any([a, b])` combina señales: se cancela con la primera.

Promesas sin atrapar

Una promesa rechazada que nadie maneja **termina el proceso** (desde Node 15). En un servidor, eso es la app caída:

src/demos/node/01-node/async/sin-catch.ts

```ts
async function guardarPostulacion() {
  throw new Error('La base de datos no responde')
}

// Una promesa rechazada que nadie atrapa (falta el await o el .catch)
guardarPostulacion()

setTimeout(() => console.log('Esto no llega a imprimirse'), 100)
```

$ node sin-catch.ts · Node 24.22.1

```
file://~/guia-react/src/demos/node/01-node/async/sin-catch.ts:2
  throw new Error('La base de datos no responde')
        ^

Error: La base de datos no responde
    at guardarPostulacion (file://~/guia-react/src/demos/node/01-node/async/sin-catch.ts:2:9)
    at file://~/guia-react/src/demos/node/01-node/async/sin-catch.ts:6:1

Node.js v24.22.1
```

```
El proceso terminó con código 1
```

Las causas típicas: llamar una función async **sin** `await` (ni `.catch`), o un `.then` sin `.catch` al final. En Express 5, los errores de una ruta async los atrapa Express (lo vemos en middlewares); fuera de las rutas, es tu responsabilidad.

> ⚠️ **forEach no espera**
>
> `array.forEach(async (x) => { await guardar(x) })` **no espera** a nadie: dispara todas las funciones y sigue. Para hacerlo en serie, `for (const x of array) await guardar(x)`. En paralelo, `await Promise.all(array.map(guardar))`.

> 🎤 **En una entrevista**
>
> "¿Qué diferencia hay entre Promise.all y Promise.allSettled?" → "Promise.all espera a que terminen todas y falla apenas falla una; lo uso cuando necesito todos los resultados y un error invalida el conjunto. Promise.allSettled espera a todas y nunca falla: devuelve para cada una si se cumplió o se rechazó, y lo uso cuando quiero procesar lo que salió bien aunque algo falle, por ejemplo mandar 50 mails y reportar cuáles fallaron."

Capítulo 5

Archivos, rutas y streams

node:path, node:fs/promises, streams con pipeline y Buffer.

Leer y escribir archivos, armar rutas y procesar datos grandes sin llenar la memoria. Son las APIs que más vas a usar en scripts y en el backend.

Rutas con node:path

src/demos/node/01-node/archivos/rutas.ts

```
import path from 'node:path'

// Armar rutas con path (y no concatenando strings): usa el separador
  correcto en cada sistema
console.log(path.join('datos', 'ofertas', '..', 'empresas.json'))
console.log(path.basename('/home/ana/cv/cv-2026.pdf'))
console.log(path.extname('cv-2026.pdf'))
console.log(path.dirname('/home/ana/cv/cv-2026.pdf'))
console.log(path.parse('/home/ana/cv/cv-2026.pdf'))

// La carpeta de este archivo, para leer archivos que están al lado
console.log(import.meta.dirname)
console.log(path.join(import.meta.dirname, 'datos.json'))
```

$ node rutas.ts · Node 24.22.1

```
datos/empresas.json
cv-2026.pdf
.pdf
/home/ana/cv
{
  root: '/',
  dir: '/home/ana/cv',
```

```
  base: 'cv-2026.pdf',
  ext: '.pdf',
  name: 'cv-2026'
}
~/guia-react/src/demos/node/01-node/archivos
~/guia-react/src/demos/node/01-node/archivos/datos.json
```

- `path.join` une partes con el separador del sistema (`/` en Mac y Linux, `\` en Windows) y resuelve los `..`. Nunca concatenes rutas con `+`.
- `path.resolve` devuelve una ruta absoluta.
- Las rutas relativas (`'datos.json'`) se resuelven contra **la carpeta desde donde ejecutaste node** (`process.cwd()`), no contra la del archivo. Para leer un archivo que está al lado del código, usá `import.meta.dirname`.

Archivos con node:fs/promises

src/demos/node/01-node/archivos/archivos.ts

```ts
import { appendFile, mkdtemp, readdir, readFile, rm, stat, writeFile }
  from 'node:fs/promises'
import { tmpdir } from 'node:os'
import { join } from 'node:path'

// Una carpeta temporal para no ensuciar el proyecto
const carpeta = await mkdtemp(join(tmpdir(), 'postulaciones-'))

const postulaciones = [
  { empresa: 'Nubelia', estado: 'entrevista' },
  { empresa: 'Pagos Sur', estado: 'oferta' },
]
await writeFile(join(carpeta, 'postulaciones.json'),
  JSON.stringify(postulaciones, null, 2))
await writeFile(join(carpeta, 'notas.txt'), 'Preparar live coding\n')
await appendFile(join(carpeta, 'notas.txt'), 'Repasar el event
  loop\n')

const leidas = JSON.parse(await readFile(join(carpeta,
  'postulaciones.json'), 'utf8'))
console.log('Postulaciones:', leidas.map((p: { empresa: string }) =>
  p.empresa))
```

```
console.log('Notas:\n' + (await readFile(join(carpeta, 'notas.txt'),
  'utf8')))

for (const nombre of (await readdir(carpeta)).sort()) {
  const info = await stat(join(carpeta, nombre))
  console.log(`${nombre}: ${info.size} bytes`)
}

await rm(carpeta, { recursive: true }) // borrar la carpeta con todo
  lo que tiene
```

$ node archivos.ts · Node 24.22.1

```
Postulaciones: [ 'Nubelia', 'Pagos Sur' ]
Notas:
Preparar live coding
Repasar el event loop

notas.txt: 43 bytes
postulaciones.json: 124 bytes
```

Función	Para
`readFile(ruta, 'utf8')`	Leer un archivo entero como texto (sin `'utf8'` devuelve un `Buffer`)
`writeFile(ruta, datos)`	Crear o reemplazar un archivo
`appendFile(ruta, datos)`	Agregar al final
`mkdir(ruta, { recursive: true })`	Crear una carpeta (y las que falten en el camino)
`readdir(ruta)`	Listar una carpeta
`stat(ruta)`	Tamaño, fechas, si es carpeta…
`rm(ruta, { recursive: true, force: true })`	Borrar un archivo o una carpeta entera
`rename(de, a)` / `copyFile(de, a)`	Mover y copiar
`access(ruta)`	Ver si existe (o, mejor, intentar leerlo y manejar el error `ENOENT`)

Los errores traen un `code`: `ENOENT` (no existe), `EACCES` (sin permiso), `EEXIST` (ya existe), `EISDIR` (es una carpeta).

> **ℹ️ Sync, callbacks y promesas**
>
> Cada función existe en tres versiones: `fs.readFileSync` (bloquea), `fs.readFile` con callback y `fsPromises.readFile`. En un servidor, la de promesas. Las `Sync` están bien en un script de línea de comandos o al arrancar la app (leer una configuración, por ejemplo).

Streams: datos de a pedazos

`readFile` carga el archivo **entero en memoria**. Con un log de 4 GB, el proceso se queda sin memoria. Un **stream** procesa los datos **de a pedazos** (*chunks*) a medida que llegan: la memoria usada es la de un pedazo, no la del archivo.

`src/demos/node/01-node/archivos/streams.ts`

```
import { createReadStream, createWriteStream } from 'node:fs'
import { mkdtemp, rm, stat } from 'node:fs/promises'
import { tmpdir } from 'node:os'
import { join } from 'node:path'
import { createInterface } from 'node:readline'
import { Readable, Transform } from 'node:stream'
import { pipeline } from 'node:stream/promises'
import { createGzip } from 'node:zlib'

const carpeta = await mkdtemp(join(tmpdir(), 'streams-'))
const csv = join(carpeta, 'visitas.csv')

// 1) Escribir un archivo grande de a pedazos, sin armarlo entero en
  memoria
function* filas() {
  yield 'fecha,pagina\n'
  for (let i = 0; i < 200_000; i++) yield `2026-10-${String((i % 30) +
    1).padStart(2, '0')},/ofertas/${i % 15}\n`
}
await pipeline(Readable.from(filas()), createWriteStream(csv))
console.log(`CSV: ${((await stat(csv)).size / 1024 / 1024).toFixed(1)}
  MB`)

// 2) Leerlo línea por línea: en memoria hay un pedazo por vez, no el
  archivo entero
let visitasOferta3 = 0
```

```
for await (const linea of createInterface({ input:
  createReadStream(csv) })) {
  if (linea.endsWith('/ofertas/3')) visitasOferta3++
}
console.log('Visitas a la oferta 3:', visitasOferta3)

// 3) Una cadena de streams: leer → transformar → comprimir → escribir
const aMayusculas = new Transform({
  transform(pedazo, _codificacion, listo) {
    listo(null, pedazo.toString().toUpperCase())
  },
})
await pipeline(createReadStream(csv), aMayusculas, createGzip(),
  createWriteStream(`${csv}.gz`))
console.log(`Comprimido: ${((await stat(`${csv}.gz`)).size /
  1024).toFixed(0)} KB`)

await rm(carpeta, { recursive: true })
```

$ node streams.ts · Node 24.22.1

```
CSV: 4.3 MB
Visitas a la oferta 3: 13334
Comprimido: 21 KB
```

Los cuatro tipos:

Tipo	Qué es	Ejemplos
Readable	Una fuente de datos	`createReadStream`, el `req` de un servidor HTTP, `Readable.from(iterable)`
Writable	Un destino	`createWriteStream`, el `res` de un servidor HTTP, `process.stdout`
Duplex	Las dos cosas	Un socket TCP
Transform	Un Duplex que transforma lo que pasa	`createGzip`, un parser de CSV, cifrado

- **`pipeline`** conecta streams en cadena, propaga los errores de cualquiera de ellos y cierra todo al final. Usalo siempre en lugar del viejo `.pipe()`, que no maneja errores.

- `for await (const pedazo of stream)`: los Readable son iterables asíncronos. Con `readline`, de a líneas.
- **Backpressure**: si el destino es más lento que la fuente (escribir en un disco lento, un cliente con mala conexión), el stream frena la lectura hasta que el destino se pone al día. `pipeline` lo maneja solo.

En un servidor, los streams aparecen en todos lados: el cuerpo de un pedido llega como stream (ver HTTP sin frameworks), y para devolver un archivo grande se hace `pipeline(createReadStream(ruta), res)` en lugar de leerlo entero.

Buffer

Un `Buffer` son bytes crudos (el tipo de Node para datos binarios, parecido a un `Uint8Array`). Aparece al leer archivos sin codificación, con imágenes, hashes y criptografía:

```
const b = Buffer.from('Canción', 'utf8')
b.length                  // 8: la "ó" ocupa 2 bytes en UTF-8
b.toString('hex')         // '43616e6369c3b36e'
b.toString('base64')      // 'Q2FuY2nDs24='
```

> 🎤 **En una entrevista**
>
> "¿Qué es un stream y cuándo lo usarías?" → "Una forma de procesar datos de a pedazos a medida que llegan, en lugar de cargar todo en memoria. Lo uso para archivos grandes (leer un CSV de varios GB línea por línea, comprimir, subir a S3), para devolver archivos en una respuesta HTTP o para hacer de proxy. Se conectan con pipeline, que maneja los errores y el backpressure: si el destino es más lento, frena la lectura."

Capítulo 6

Eventos y el proceso

EventEmitter, argumentos, variables de entorno, códigos de salida y señales.

El objeto global `process` es la conexión de tu programa con el sistema: los argumentos, las variables de entorno, la salida, las señales. Y `EventEmitter` es el patrón de eventos sobre el que están construidas muchas APIs de Node.

EventEmitter

Un objeto que **emite eventos** con nombre, y cualquiera puede **suscribirse**. Lo usan los servidores HTTP (`'request'`, `'close'`), los streams (`'data'`, `'end'`, `'error'`), los procesos y los workers.

src/demos/node/01-node/proceso/eventos.ts

```
import { EventEmitter, once } from 'node:events'

type Postulacion = { empresa: string; estado: string }

// Los eventos que emite, con el tipo de sus argumentos
type Eventos = {
  nueva: [postulacion: Postulacion]
  cambio: [postulacion: Postulacion, anterior: string]
  error: [error: Error]
}

class Postulaciones extends EventEmitter<Eventos> {
  #lista: Postulacion[] = []

  agregar(empresa: string) {
    const postulacion = { empresa, estado: 'postulado' }
    this.#lista.push(postulacion)
```

```
    this.emit('nueva', postulacion)
  }

  avanzar(empresa: string, estado: string) {
    const postulacion = this.#lista.find((p) => p.empresa === empresa)
    if (!postulacion) return this.emit('error', new Error(`No hay
      postulación a ${empresa}`))
    const anterior = postulacion.estado
    postulacion.estado = estado
    this.emit('cambio', postulacion, anterior)
  }
}

const postulaciones = new Postulaciones()

// Varios oyentes para el mismo evento, cada uno con lo suyo
postulaciones.on('nueva', (p) => console.log(`Nueva postulación:
  ${p.empresa}`))
postulaciones.on('cambio', (p, anterior) => console.log(`${p.empresa}:
  ${anterior} → ${p.estado}`))
postulaciones.once('cambio', () => console.log('(once: este oyente se
  ejecuta solo la primera vez)'))
// Sin un oyente de 'error', emitir 'error' tira una excepción y corta
  el proceso
postulaciones.on('error', (error) => console.log('Error:',
  error.message))

postulaciones.agregar('Nubelia')
postulaciones.avanzar('Nubelia', 'entrevista')
postulaciones.avanzar('Nubelia', 'oferta')
postulaciones.avanzar('Pagos Sur', 'entrevista')

// once() de node:events devuelve una promesa: esperar un evento con
  await
setTimeout(() => postulaciones.agregar('Delta Pay'), 50)
const [nueva] = await once(postulaciones, 'nueva')
console.log('Llegó con await:', nueva.empresa)
```

$ node eventos.ts · Node 24.22.1

```
Nueva postulación: Nubelia
Nubelia: postulado → entrevista
(once: este oyente se ejecuta solo la primera vez)
Nubelia: entrevista → oferta
Error: No hay postulación a Pagos Sur
Nueva postulación: Delta Pay
Llegó con await: Delta Pay
```

- `on(evento, fn)` suscribe, `once` suscribe para una sola vez y `off` desuscribe.
- `emit(evento, ...args)` llama a los oyentes **en orden y de forma sincrónica**.
- El evento `'error'` es especial: si se emite y nadie lo escucha, se lanza como excepción.
- Pasándole un tipo (`EventEmitter<Eventos>`), TypeScript chequea los nombres de los eventos y sus argumentos.

> ⚠️ **Oyentes que se acumulan**
>
> Cada `on` agrega un oyente que vive mientras viva el emisor. Suscribirse adentro de una función que se llama muchas veces (por ejemplo, en cada pedido) sin hacer `off` es una **pérdida de memoria** clásica. Node avisa con `MaxListenersExceededWarning` cuando un evento pasa los 10 oyentes.

Argumentos de la línea de comandos

`src/demos/node/01-node/proceso/argumentos.ts`

```ts
import { parseArgs } from 'node:util'

// Lo que se escribió después de "node argumentos.ts"
console.log('process.argv:', process.argv.slice(2))

// parseArgs: las opciones de la línea de comandos, con tipos y
  valores por defecto
const { values, positionals } = parseArgs({
  allowPositionals: true,
  options: {
    seniority: { type: 'string', short: 's', default: 'semi' },
    remoto: { type: 'boolean', default: false },
    minimo: { type: 'string' },
  },
})
console.log('Opciones:', values)
console.log('Palabras clave:', positionals)
```

$ node argumentos.ts react node -s senior --remoto --minimo 3000 · Node 24.22.1

```
process.argv: [
  'react',    'node',
  '-s',       'senior',
  '--remoto', '--minimo',
  '3000'
]
Opciones: [Object: null prototype] {
  seniority: 'senior',
  remoto: true,
  minimo: '3000'
}
Palabras clave: [ 'react', 'node' ]
```

`process.argv` tiene todo como texto: en la posición 0 está la ruta de `node`, en la 1 la del archivo, y después lo que escribiste. `parseArgs` (de `node:util`) los interpreta con tipos y valores por defecto, sin instalar nada. Para una herramienta de línea de comandos más completa existen **commander** o **yargs**.

Variables de entorno

La configuración que cambia según dónde corre la app (la base de datos, las claves, el puerto) **no va en el código**: va en variables de entorno.

`src/demos/node/01-node/proceso/entorno.ts`

```ts
// Las variables de entorno siempre son strings (o undefined si no
  están)
const puerto = Number(process.env.PUERTO ?? 3000)
console.log('PUERTO:', puerto, typeof puerto)
console.log('DB_NAME:', process.env.DB_NAME)
console.log('NO_EXISTE:', process.env.NO_EXISTE)
console.log('API_KEY definida:', Boolean(process.env.API_KEY))

// Otros datos del proceso
console.log('Versión de Node:', process.version.split('.')[0])
```

$ node --env-file=.env.ejemplo entorno.ts · Node 24.22.1

```
PUERTO: 4000 number
DB_NAME: busqueda
NO_EXISTE: undefined
API_KEY definida: true
Versión de Node: v24
```

- `process.env.X` siempre es un **string** o `undefined`. `PUERTO=4000` llega como `'4000'`: convertilo.
- `node --env-file=.env` carga un archivo `.env` (desde Node 20.6; antes se usaba el paquete **dotenv**). `--env-file-if-exists` no falla si el archivo no está. Desde el código: `process.loadEnvFile()`.
- El `.env` va en el `.gitignore`. Al repositorio se sube un `.env.example` con los nombres de las variables y valores de mentira.
- En producción vemos cómo validar todas las variables al arrancar.

Código de salida

Cuando un proceso termina, le devuelve un número al sistema: **0** si salió bien, **otro** si falló. Lo leen la terminal (`echo $?`), los scripts de npm, el CI (un test que falla devuelve 1 y el pipeline se corta) y Docker.

`src/demos/node/01-node/proceso/codigo-de-salida.ts`

```ts
const archivos = process.argv.slice(2)

if (archivos.length === 0) {
  console.error('Uso: node codigo-de-salida.ts <archivo>...')
  // 0 = todo bien; cualquier otro número = algo falló (lo leen los
    scripts, CI, Docker…)
  process.exitCode = 1
} else {
  console.log('Procesando', archivos)
```

```
}
```

$ node codigo-de-salida.ts · Node 24.22.1

```
Uso: node codigo-de-salida.ts <archivo>...
```

El proceso terminó con código 1

`process.exitCode = 1` marca el código y deja que el proceso termine solo cuando no quede nada pendiente. `process.exit(1)` corta **en el acto**, aunque haya escrituras o pedidos a medio terminar: usalo solo cuando no hay otra opción.

Señales y errores no atrapados

```
// Ctrl+C en la terminal manda SIGINT; Docker y Kubernetes mandan
  SIGTERM antes de apagar
process.on('SIGTERM', () => apagarProlijo())
process.on('SIGINT', () => apagarProlijo())

// Último recurso: registrar el error y salir (el proceso quedó en un
  estado desconocido)
process.on('uncaughtException', (error) => {
  logger.fatal(error)
  process.exit(1)
})
process.on('unhandledRejection', (motivo) => {
  logger.fatal(motivo)
  process.exit(1)
})
```

Un `uncaughtException` **no** se usa para "seguir como si nada": después de un error inesperado, el estado de la app es dudoso. Se registra el error, se sale, y el orquestador (Docker, PM2, Kubernetes) levanta un proceso nuevo. El apagado prolijo con `SIGTERM` lo vemos en producción.

🎤 En una entrevista

"¿Cómo manejás la configuración en una app de Node?" → "Con variables de entorno, nunca en el código: en desarrollo, un .env que carga node --env-file (o dotenv) y que no se sube al repo; en producción, las variables del servidor o los secretos del proveedor. Al arrancar las valido todas juntas, por ejemplo con Zod, y si falta alguna la app no arranca, con un mensaje claro. Y recuerdo que process.env siempre devuelve strings."

Capítulo 7

HTTP sin frameworks

Un servidor con node:http, métodos y status codes, y fetch como cliente.

Antes de Express, un servidor con el módulo `node:http`, sin ninguna librería. No es lo que vas a escribir en un proyecto, pero muestra qué hace Express por vos.

Un servidor con node:http

src/demos/node/01-node/http/servidor.ts

```
import { createServer, type IncomingMessage, type ServerResponse }
  from 'node:http'

const ofertas = [
  { id: 1, titulo: 'Frontend React Senior', modalidad: 'remoto' },
  { id: 2, titulo: 'Fullstack Node', modalidad: 'remoto' },
]

function responderJson(res: ServerResponse, status: number, datos:
  unknown) {
  res.writeHead(status, { 'Content-Type': 'application/json' })
  res.end(JSON.stringify(datos))
}

// El cuerpo de un pedido llega en pedazos (es un stream): hay que
  juntarlos
async function leerCuerpo(req: IncomingMessage) {
  let texto = ''
  for await (const pedazo of req) texto += pedazo
  return JSON.parse(texto)
}

// Una sola función atiende TODOS los pedidos: el "ruteo" es a mano
export const app = createServer(async (req, res) => {
```

```
  const url = new URL(req.url ?? '/', 'http://localhost')
  console.log(req.method, url.pathname)

  if (req.method === 'GET' && url.pathname === '/ofertas') {
    return responderJson(res, 200, ofertas)
  }
  const id = url.pathname.match(/^\/ofertas\/(\d+)$/)?.[1]
  if (req.method === 'GET' && id) {
    const oferta = ofertas.find((o) => o.id === Number(id))
    return oferta ? responderJson(res, 200, oferta) :
      responderJson(res, 404, { error: 'No existe' })
  }
  if (req.method === 'POST' && url.pathname === '/ofertas') {
    try {
      const nueva = { id: ofertas.length + 1, ...(await
        leerCuerpo(req)) }
      ofertas.push(nueva)
      return responderJson(res, 201, nueva)
    } catch {
      return responderJson(res, 400, { error: 'JSON inválido' })
    }
  }
  responderJson(res, 404, { error: `No existe ${req.method}
    ${url.pathname}` })
})

// Solo si se ejecuta este archivo (node servidor.ts), no cuando otro
  lo importa
if (import.meta.main) {
  app.listen(3000, () => console.log('Escuchando en
    http://localhost:3000'))
}
```

PEDIDOS A LA APP · Node 24.22.1 · Express 5.3.0

GET `/ofertas`

← 200 OK

```
[
  {
    "id": 1,
    "titulo": "Frontend React Senior",
    "modalidad": "remoto"
  },
  {
    "id": 2,
    "titulo": "Fullstack Node",
    "modalidad": "remoto"
  }
]
```

CONSOLA DE LA APP

```
GET /ofertas
```

GET `/ofertas/2`

← 200 OK

```
{
  "id": 2,
  "titulo": "Fullstack Node",
  "modalidad": "remoto"
}
```

CONSOLA DE LA APP

```
GET /ofertas/2
```

GET `/ofertas/9`

← 404 Not Found

```
{
  "error": "No existe"
}
```

CONSOLA DE LA APP

```
GET /ofertas/9
```

POST `/ofertas`

```
{
  "titulo": "Backend Node",
  "modalidad": "remoto"
}
```

← 201 Created

```
{
  "id": 3,
  "titulo": "Backend Node",
  "modalidad": "remoto"
}
```

CONSOLA DE LA APP

```
POST /ofertas
```

POST `/ofertas`

```
{"titulo": "Mal escrito",
```

← 400 Bad Request

```
{
  "error": "JSON inválido"
}
```

CONSOLA DE LA APP

```
POST /ofertas
```

DELETE `/ofertas/1`

← 404 Not Found

```
{
  "error": "No existe DELETE /ofertas/1"
}
```

CONSOLA DE LA APP

```
DELETE /ofertas/1
```

- `createServer` recibe **una** función que atiende **todos** los pedidos. `req` (un `IncomingMessage`) es lo que llegó: método, URL, headers y el cuerpo. `res` (un `ServerResponse`) es lo que vas a responder.
- El ruteo es a mano: comparar el método y la URL, sacar el `:id` con una expresión regular.
- El cuerpo llega **como stream**, en pedazos: hay que juntarlos y parsear el JSON (y manejar el error si viene mal).
- Para responder JSON hay que poner el `Content-Type` y hacer `JSON.stringify`.
- `if (import.meta.main)` levanta el servidor solo cuando ejecutás este archivo (`node servidor.ts`). Si otro archivo lo importa (un test, o el script que generó estas respuestas), no.

Todo eso, más middlewares, manejo de errores y mucho más, es lo que resuelve Express en el capítulo siguiente.

HTTP en dos minutos

Un pedido HTTP tiene un **método**, una **ruta**, **headers** y, a veces, un **cuerpo**. La respuesta tiene un **código de estado**, headers y un cuerpo.

Método	Para	¿Tiene cuerpo?	¿Idempotente?
`GET`	Leer	No	Sí
`POST`	Crear, o una acción	Sí	No
`PUT`	Reemplazar entero	Sí	Sí
`PATCH`	Cambiar una parte	Sí	No necesariamente
`DELETE`	Borrar	No	Sí

Idempotente: repetir el pedido deja todo igual que hacerlo una vez. Si un `DELETE /ofertas/1` se manda dos veces por un reintento, la oferta sigue borrada; dos `POST` crean dos ofertas.

Los códigos que vas a usar: **200** OK, **201** Created, **204** No Content, **301/302** redirección, **304** Not Modified, **400** Bad Request (datos

inválidos), **401** Unauthorized (no sé quién sos), **403** Forbidden (sé quién sos y no podés), **404** Not Found, **409** Conflict, **422** Unprocessable Content, **429** Too Many Requests, **500** Internal Server Error, **502/503** el servidor de atrás no responde o no está disponible.

fetch como cliente

Node trae `fetch` incluido, con la misma API que el navegador. Sirve para llamar a otras APIs desde tu backend:

src/demos/node/01-node/http/cliente.ts

```
import { createServer } from 'node:http'

// Un servidor de prueba, para tener a quién pedirle algo
const servidor = createServer((req, res) => {
  if (req.url === '/lento') return setTimeout(() => res.end('por
    fin'), 1000)
  if (req.url === '/ofertas') {
    res.writeHead(200, { 'Content-Type': 'application/json' })
    return res.end(JSON.stringify([{ id: 1, titulo: 'Frontend React
      Senior' }]))
  }
  res.writeHead(404).end()
}).listen(0)
const { port } = servidor.address() as { port: number }
const base = `http://localhost:${port}`

// fetch viene incluido en Node (desde la 18): la misma API que en el
  navegador
const res = await fetch(`${base}/ofertas`)
console.log(res.status, res.headers.get('content-type'))
console.log(await res.json())

// fetch NO falla con un 404 o un 500: hay que mirar res.ok
const noExiste = await fetch(`${base}/no-existe`)
console.log('ok:', noExiste.ok, '· status:', noExiste.status)

// Sin timeout, un servidor colgado te deja esperando para siempre
try {
  await fetch(`${base}/lento`, { signal: AbortSignal.timeout(200) })
} catch (error) {
  console.log('Timeout:', (error as Error).name)
}
```

```
servidor.close()
servidor.closeAllConnections()
```

$ node cliente.ts · Node 24.22.1

```
200 application/json
[ { id: 1, titulo: 'Frontend React Senior' } ]
ok: false · status: 404
Timeout: TimeoutError
```

⚠️ Los tres errores típicos con fetch

1. `fetch` **no falla con un 404 o un 500**: solo falla si no hubo respuesta (sin red, DNS, conexión rechazada). Siempre mirá `res.ok` o `res.status`.
2. **Sin timeout** espera para siempre. Pasale `signal: AbortSignal.timeout(5000)`.
3. `res.json()` **devuelve una promesa** (y `unknown` en TypeScript): hay que hacer `await` y validar la forma de los datos, por ejemplo con Zod.

🎤 En una entrevista

"¿Qué pasa cuando escribís una URL en el navegador y apretás Enter?" → "Se resuelve el dominio a una IP con DNS, se abre una conexión TCP (y TLS si es HTTPS) con el servidor, y se manda un pedido HTTP GET con headers. En el servidor, Node recibe el pedido, el framework lo pasa por los middlewares y la ruta, y se arma la respuesta con un status, headers y un cuerpo (HTML, JSON). El navegador la recibe, parsea el HTML, pide los recursos que referencia (CSS, JS, imágenes), arma el DOM y pinta la página."

Parte 2 · Express

APIs con Express 5: de la primera ruta a una app en capas, segura, testeada y lista para producción.

Capítulo 8

Express: rutas y respuestas

La app, req y res, params, query y body, Router y cómo organizar el proyecto.

Express es un framework mínimo para servidores HTTP: rutas, middlewares y helpers para leer pedidos y armar respuestas. Es el más usado de Node, el que más vas a ver en ofertas laborales, y la base de muchos otros.

```
npm install express
npm install -D @types/express
```

La app más chica

src/demos/node/02-express/hola.ts

```ts
import express from 'express'

export const app = express()

// Una ruta = método HTTP + camino + función que responde
app.get('/', (_req, res) => {
  res.send('Hola desde Express')
})

app.get('/salud', (_req, res) => {
  res.json({ ok: true, version: '1.0.0' })
```

```
})

// Solo si se ejecuta este archivo (node hola.ts); los tests lo
  importan sin levantarlo
if (import.meta.main) {
  app.listen(3000, () => console.log('Escuchando en
    http://localhost:3000'))
}
```

PEDIDOS A LA APP · Node 24.22.1 · Express 5.3.0

GET `/`

← 200 OK

```
Hola desde Express
```

GET `/salud`

← 200 OK

```
{
  "ok": true,
  "version": "1.0.0"
}
```

GET `/no-existe`

← 404 Not Found

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>Cannot GET /no-existe</pre>
</body>
</html>
```

- `express()` crea la app. `app.get(ruta, handler)` registra qué hacer con un `GET` a esa ruta; hay `app.post`, `app.put`, `app.patch`, `app.delete` y `app.all`.
- El handler recibe `req` (el pedido) y `res` (la respuesta). Una ruta tiene que responder siempre, o el cliente se queda esperando.

- Si ninguna ruta coincide, Express responde un 404 en HTML. En middlewares lo cambiamos por uno en JSON.

Rutas, parámetros y Router

src/demos/node/02-express/rutas.ts

```ts
import express from 'express'
import { ofertas } from './datos.ts'

export const app = express()
app.use(express.json()) // lee los cuerpos JSON y los deja en req.body

// Un Router agrupa las rutas de un recurso; después se "monta" en un
  camino
const rutasOfertas = express.Router()

// GET /api/ofertas?modalidad=remoto&q=front
rutasOfertas.get('/', (req, res) => {
  const { modalidad, q } = req.query // lo que va después del ?
  let resultado = ofertas
  if (typeof modalidad === 'string') resultado = resultado.filter((o)
    => o.modalidad === modalidad)
  if (typeof q === 'string') {
    resultado = resultado.filter((o) =>
      o.titulo.toLowerCase().includes(q.toLowerCase()))
  }
  res.json(resultado)
})

// GET /api/ofertas/3: los :parametros de la ruta llegan en req.params
  (siempre como string)
rutasOfertas.get('/:id', (req, res) => {
  const oferta = ofertas.find((o) => o.id === Number(req.params.id))
  if (!oferta) return res.status(404).json({ error: 'Oferta no
    encontrada' })
  res.json(oferta)
})

// POST /api/ofertas con un JSON en el cuerpo
rutasOfertas.post('/', (req, res) => {
  const nueva = { id: ofertas.length + 1, ...req.body }
  ofertas.push(nueva)
  res.status(201).location(`/api/ofertas/${nueva.id}`).json(nueva)
})
```

```
app.use('/api/ofertas', rutasOfertas)
```

src/demos/node/02-express/datos.ts

```
// Datos de ejemplo en memoria (cada ejemplo arranca con estos mismos)
export type Modalidad = 'remoto' | 'hibrido' | 'presencial'

export type Oferta = {
  id: number
  titulo: string
  empresa: string
  modalidad: Modalidad
  salarioMax: number | null
}

export const ofertas: Oferta[] = [
  { id: 1, titulo: 'Frontend React Senior', empresa: 'Nubelia',
    modalidad: 'remoto', salarioMax: 4000 },
  { id: 2, titulo: 'Fullstack Node', empresa: 'Nubelia', modalidad:
    'remoto', salarioMax: 3000 },
  { id: 3, titulo: 'React Native Developer', empresa: 'Pagos Sur',
    modalidad: 'hibrido', salarioMax: 3200 },
  { id: 4, titulo: 'Frontend Junior', empresa: 'Mate Cloud',
    modalidad: 'presencial', salarioMax: 1600 },
  { id: 5, titulo: 'Senior Frontend Engineer', empresa: 'Delta Pay',
    modalidad: 'remoto', salarioMax: 6000 },
]
```

PEDIDOS A LA APP · Node 24.22.1 · Express 5.3.0

GET `/api/ofertas?modalidad=remoto`

← 200 OK


```
[
  {
    "id": 1,
    "titulo": "Frontend React Senior",
    "empresa": "Nubelia",
    "modalidad": "remoto",
    "salarioMax": 4000
  },
  {
    "id": 2,
    "titulo": "Fullstack Node",
    "empresa": "Nubelia",
    "modalidad": "remoto",
    "salarioMax": 3000
  },
  {
    "id": 5,
    "titulo": "Senior Frontend Engineer",
    "empresa": "Delta Pay",
    "modalidad": "remoto",
    "salarioMax": 6000
  }
]
```

GET `/api/ofertas?q=front`

← 200 OK

```
[
  {
    "id": 1,
    "titulo": "Frontend React Senior",
    "empresa": "Nubelia",
    "modalidad": "remoto",
    "salarioMax": 4000
  },
  {
    "id": 4,
    "titulo": "Frontend Junior",
    "empresa": "Mate Cloud",
    "modalidad": "presencial",
    "salarioMax": 1600
  },
  {
    "id": 5,
    "titulo": "Senior Frontend Engineer",
    "empresa": "Delta Pay",
    "modalidad": "remoto",
    "salarioMax": 6000
  }
]
```

GET `/api/ofertas/3`

← 200 OK

```
{
  "id": 3,
  "titulo": "React Native Developer",
  "empresa": "Pagos Sur",
  "modalidad": "hibrido",
  "salarioMax": 3200
}
```

GET `/api/ofertas/99`

← 404 Not Found

```
{
  "error": "Oferta no encontrada"
}
```

POST `/api/ofertas`

```
{
  "titulo": "Backend Node",
  "empresa": "Delta Pay",
  "modalidad": "remoto",
  "salarioMax": 4500
}
```

← 201 Created

```
location: /api/ofertas/6
```

```
{
  "id": 6,
  "titulo": "Backend Node",
  "empresa": "Delta Pay",
  "modalidad": "remoto",
  "salarioMax": 4500
}
```

Lo que llega	Dónde	Ejemplo
Parámetros de la ruta	`req.params`	`/ofertas/:id` → `req.params.id` es `'3'` (siempre string)
Query string	`req.query`	`?modalidad=remoto&q=front` → `{ modalidad: 'remoto', q: 'front' }`
Cuerpo	`req.body`	Solo si un middleware lo parseó: `app.use(express.json())`
Headers	`req.get('authorization')` o `req.headers`	Los nombres en minúscula
Otros	`req.method`, `req.path`, `req.originalUrl`, `req.ip`, `req.cookies` (con `cookie-parser`)	

Para responder	
`res.json(datos)`	JSON con su `Content-Type`

Para responder	
`res.send(texto)`	Texto, HTML o un Buffer
`res.status(201)`	El código (se encadena: `res.status(201).json(...)`)
`res.set('X-Header', 'valor')`, `res.location(url)`	Headers
`res.redirect('/otra')`	Redirección (302 por defecto)
`res.status(204).end()`	Sin cuerpo
`res.sendFile(ruta)`, `res.download(ruta)`	Archivos

`express.Router()` agrupa las rutas de un recurso en su propio archivo y se monta con `app.use('/api/ofertas', router)`: adentro del router, las rutas son relativas (`'/'`, `'/:id'`).

> ⚠️ **Lo que viene en el pedido no es confiable**
>
> `req.query`, `req.params` y `req.body` son lo que mandó **el cliente**: pueden faltar campos, venir con otro tipo o traer cosas de más. En el `POST` del ejemplo, `{ id: ..., ...req.body }` copia cualquier cosa que manden. Se arregla validando, en el capítulo de validación.

Cómo organizar un proyecto

```
src/
  app.ts              Crea la app de Express, registra middlewares y
    rutas (sin listen)
  servidor.ts         Importa la app y hace listen: el punto de
    entrada
  config.ts           Variables de entorno validadas
  ofertas/
    ofertas.rutas.ts      El Router del recurso
    ofertas.servicio.ts   Las reglas del negocio
    ofertas.repositorio.ts  El acceso a la base de datos
    ofertas.esquemas.ts   Los esquemas de Zod
  middlewares/        Autenticación, manejo de errores…
```

```
test/
```

Separar `app.ts` (que exporta la app) de `servidor.ts` (que la levanta) permite importarla en los tests sin abrir un puerto. En la guía, cada ejemplo hace lo mismo en un solo archivo con `if (import.meta.main)`. La organización por capas está en arquitectura.

> **ℹ️ ¿Express o algo más nuevo?**
>
> Express es el estándar de hecho y lo vas a encontrar en la mayoría de los proyectos. Alternativas que vas a ver en ofertas: **Fastify** (más rápido, con validación por esquemas incluida), **NestJS** (estructurado con clases y decoradores, estilo Angular, usa Express o Fastify por debajo) y **Hono** (muy liviano, corre en Node, Bun, Deno y en el edge). Los conceptos de esta guía (rutas, middlewares, capas, validación) se trasladan a todos.

> **🎤 En una entrevista**
>
> "¿Qué diferencia hay entre req.params, req.query y req.body?" → "params son los segmentos variables de la ruta, como el :id de /ofertas/:id; query es lo que va después del signo de pregunta, para filtros, orden y paginación; body es el cuerpo del pedido, el JSON de un POST o un PUT, y Express solo lo llena si hay un middleware como express.json(). Los tres vienen del cliente, así que se validan siempre, y params y query siempre llegan como string."

Capítulo 9

Middlewares

La cadena del pedido, next(), errores en Express 5, 404 y CORS.

Un **middleware** es una función que se ejecuta **en el camino** entre que llega el pedido y la ruta que responde. Express es, básicamente, una cadena de middlewares: `express.json()` es uno, `cors()` es otro, y una ruta es un middleware que responde.

```
function miMiddleware(req, res, next) {
  // 1. Hacer algo con el pedido (leer un header, medir, verificar)
  // 2. O responder y cortar la cadena: res.status(401).json(...)
  // 3. O pasarle el pedido al siguiente:
  next()
}
```

La cadena en acción

src/demos/node/02-express/middlewares.ts

```
import express, { type ErrorRequestHandler, type RequestHandler } from
  'express'
import { ofertas } from './datos.ts'

export const app = express()

// 1) Para todos los pedidos: loguear cada uno cuando termina
app.use((req, res, next) => {
  res.on('finish', () => console.log(`${req.method} ${req.originalUrl}
    → ${res.statusCode}`))
  next() // sin esto, el pedido se queda colgado acá
})
app.use(express.json())
```

```ts
// 2) Solo para algunas rutas: pedir una API key
const requiereApiKey: RequestHandler = (req, res, next) => {
  if (req.get('x-api-key') !== 'clave-de-prueba') {
    return res.status(401).json({ error: 'Falta la API key o es
      incorrecta' })
  }
  next()
}

app.get('/ofertas', (_req, res) => {
  res.json(ofertas.map((o) => o.titulo))
})

app.delete('/ofertas/:id', requiereApiKey, (req, res) => {
  const i = ofertas.findIndex((o) => o.id === Number(req.params.id))
  if (i === -1) return res.status(404).json({ error: 'Oferta no
    encontrada' })
  ofertas.splice(i, 1)
  res.status(204).end()
})

// 3) Una ruta async que falla: Express 5 manda el error al manejador
  de errores solo
app.get('/postulaciones', async () => {
  throw new Error('La base de datos no responde')
})

// 4) Si ninguna ruta respondió: 404
app.use((req, res) => {
  res.status(404).json({ error: `No existe ${req.method} ${req.path}`
    })
})

// 5) El manejador de errores: 4 parámetros, y va al final de todo
const manejarErrores: ErrorRequestHandler = (error, _req, res, _next)
  => {
  const status = error.status ?? 500 // express.json() usa 400 para un
    JSON mal escrito
  console.error(`Error ${status}: ${error.message}`)
  res.status(status).json({ error: status === 500 ? 'Error interno' :
    error.message })
}
app.use(manejarErrores)
```

src/demos/node/02-express/datos.ts

```ts
// Datos de ejemplo en memoria (cada ejemplo arranca con estos mismos)
export type Modalidad = 'remoto' | 'hibrido' | 'presencial'
```

```ts
export type Oferta = {
  id: number
  titulo: string
  empresa: string
  modalidad: Modalidad
  salarioMax: number | null
}

export const ofertas: Oferta[] = [
  { id: 1, titulo: 'Frontend React Senior', empresa: 'Nubelia',
    modalidad: 'remoto', salarioMax: 4000 },
  { id: 2, titulo: 'Fullstack Node', empresa: 'Nubelia', modalidad:
    'remoto', salarioMax: 3000 },
  { id: 3, titulo: 'React Native Developer', empresa: 'Pagos Sur',
    modalidad: 'hibrido', salarioMax: 3200 },
  { id: 4, titulo: 'Frontend Junior', empresa: 'Mate Cloud',
    modalidad: 'presencial', salarioMax: 1600 },
  { id: 5, titulo: 'Senior Frontend Engineer', empresa: 'Delta Pay',
    modalidad: 'remoto', salarioMax: 6000 },
]
```

PEDIDOS A LA APP · Node 24.22.1 · Express 5.3.0

GET `/ofertas`

← 200 OK

```
[
  "Frontend React Senior",
  "Fullstack Node",
  "React Native Developer",
  "Frontend Junior",
  "Senior Frontend Engineer"
]
```

CONSOLA DE LA APP

```
GET /ofertas → 200
```

DELETE `/ofertas/1`

← 401 Unauthorized

```
{
  "error": "Falta la API key o es incorrecta"
}
```

CONSOLA DE LA APP

```
DELETE /ofertas/1 → 401
```

DELETE /ofertas/1 x-api-key: clave-de-prueba

← 204 No Content

CONSOLA DE LA APP

DELETE /ofertas/1 → 204

DELETE /ofertas/1 x-api-key: clave-de-prueba

← 404 Not Found

```
{
  "error": "Oferta no encontrada"
}
```

CONSOLA DE LA APP

DELETE /ofertas/1 → 404

GET /postulaciones

← 500 Internal Server Error

```
{
  "error": "Error interno"
}
```

CONSOLA DE LA APP

Error 500: La base de datos no responde

GET /postulaciones → 500

POST /ofertas

```
{"titulo": "sin cerrar"
```

← 400 Bad Request

```
{
  "error": "Expected ',' or '}' after property value in JSON at
position 23 (line 1 column 24)"
}
```

CONSOLA DE LA APP

```
Error 400: Expected ',' or '}' after property value in JSON at
position 23 (line 1 column 24)
POST /ofertas → 400
```

GET /no-existe

← 404 Not Found

```
{
  "error": "No existe GET /no-existe"
}
```

CONSOLA DE LA APP

```
GET /no-existe → 404
```

Cada pedido recorre los `app.use` y las rutas **en el orden en que se registraron**:

1. **El logger** se registra primero para ver todos los pedidos. Escucha el evento `finish` de la respuesta para loguear el status final.
2. `express.json()` parsea el cuerpo. Si el JSON viene mal, llama a `next(error)` con un status 400.
3. `requiereApiKey` está solo en la ruta del `DELETE`: se pasan varios handlers y corren en orden. Sin la clave, responde 401 y **no llama a** `next()`: la ruta nunca se ejecuta.
4. **La ruta async que falla**: en Express 5, si una función async lanza un error (o su promesa se rechaza), Express lo pasa solo al manejador de errores.

5. **El 404**: un middleware sin ruta, al final. Si llegó hasta acá, nadie respondió.
6. **El manejador de errores** se reconoce porque tiene **cuatro parámetros** `(error, req, res, next)`. Va último.

> ⚠️ **Los errores típicos**
>
> - **Olvidarse de** `next()` (y no responder): el pedido queda colgado hasta que el cliente se cansa.
> - **Responder dos veces** (un `res.json` y después otro, por un `return` que falta): `Error [ERR_HTTP_HEADERS_SENT]: Cannot set headers after they are sent`.
> - **El orden**: un middleware registrado **después** de las rutas no corre para esas rutas. `express.json()` va antes que las rutas que leen `req.body`; el 404 y el manejador de errores, al final.

× Express 4

```
// Express 4 no atrapa errores de funciones async:
// sin try/catch, la promesa rechazada queda sin manejar
app.get('/postulaciones', async (req, res, next) => {
  try {
    res.json(await repo.listar())
  } catch (error) {
    next(error)
  }
})
```

✓ Express 5

```
// Express 5 pasa el error a next() solo
app.get('/postulaciones', async (req, res) => {
  res.json(await repo.listar())
})
```

En proyectos con Express 4 vas a ver el paquete **express-async-errors** o un wrapper `asyncHandler(fn)` que hace exactamente eso.

CORS

Si tu front (en `localhost:5173` o `mi-front.vercel.app`) llama a tu API (en `localhost:3000` o `api.midominio.com`), son **orígenes distintos**. El navegador manda el pedido, pero **no deja que el JavaScript lea la respuesta** salvo que el servidor diga que ese origen está permitido:

src/demos/node/02-express/cors.ts

```
import express from 'express'

export const app = express()

const permitidos = ['http://localhost:5173', 'https://mi-
  front.vercel.app']

// CORS a mano, para ver qué hace: el navegador manda el header Origin
  y solo deja que
// el JavaScript de esa página lea la respuesta si el servidor lo
  autoriza.
app.use((req, res, next) => {
  const origen = req.get('origin')
  if (origen && permitidos.includes(origen)) {
    res.set('Access-Control-Allow-Origin', origen)
    res.set('Vary', 'Origin')
  }
  // Antes de un PUT, DELETE o un pedido con headers propios, el
    navegador pregunta con OPTIONS
  if (req.method === 'OPTIONS') {
    res.set('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE')
    res.set('Access-Control-Allow-Headers', 'Content-Type,
      Authorization')
    return res.status(204).end()
  }
  next()
})

app.get('/ofertas', (_req, res) => {
  res.json([{ id: 1, titulo: 'Frontend React Senior' }])
})
```

PEDIDOS A LA APP · Node 24.22.1 · Express 5.3.0

GET `/ofertas` origin: http://localhost:5173

← 200 OK

```
access-control-allow-origin: http://localhost:5173
```

```
[
  {
    "id": 1,
    "titulo": "Frontend React Senior"
  }
]
```

GET `/ofertas` origin: https://sitio-malicioso.com

← 200 OK

```
[
  {
    "id": 1,
    "titulo": "Frontend React Senior"
  }
]
```

OPTIONS `/ofertas` origin: http://localhost:5173 access-control-request-method: DELETE

← 204 No Content

```
access-control-allow-origin: http://localhost:5173
access-control-allow-methods: GET, POST, PUT, DELETE
access-control-allow-headers: Content-Type, Authorization
```

- Al origen permitido le devuelve `Access-Control-Allow-Origin`. Al otro no: el servidor **igual responde**, pero el navegador bloquea la lectura. CORS es una protección **del navegador**: `curl`, Postman u otro servidor no la aplican.
- Para un `PUT`, un `DELETE` o un pedido con headers propios (`Authorization`, `Content-Type: application/json`), el navegador primero pregunta con un **preflight** `OPTIONS`.

En un proyecto se usa el paquete **cors**: `app.use(cors({ origin: ['http://localhost:5173'], credentials: true }))`. Evitá `origin: '*'` en una API con usuarios logueados.

Middlewares que vas a encontrar

Paquete	Para qué
`express.json()`, `express.urlencoded()`	Parsear el cuerpo (vienen con Express)
`express.static('public')`	Servir archivos estáticos (viene con Express)
`cors`	CORS
`helmet`	Headers de seguridad (lo vemos en autenticación)
`express-rate-limit`	Limitar pedidos por IP
`cookie-parser`	Leer cookies en `req.cookies`
`morgan`, `pino-http`	Logs de cada pedido
`multer`	Subida de archivos (`multipart/form-data`)
`compression`	Comprimir las respuestas con gzip

> 🎤 **En una entrevista**
>
> "¿Qué es un middleware en Express?" → "Una función (req, res, next) que se ejecuta en el camino del pedido, en el orden en que se registró. Puede modificar el pedido, responder y cortar la cadena, o llamar a next() para pasar al siguiente. Se usan para todo lo transversal: parsear el body, CORS, logs, autenticación, rate limiting. Los de manejo de errores tienen cuatro parámetros y van al final; en Express 5, los errores de las rutas async les llegan solos."

Capítulo 10

Diseñar una API REST

Un CRUD completo, convenciones, status codes, paginación e idempotencia.

REST es una forma de diseñar APIs sobre HTTP: los datos son **recursos** con una URL (`/ofertas` , `/ofertas/3`), y los **métodos** HTTP dicen qué hacer con ellos. No es un estándar estricto sino un conjunto de convenciones, y seguirlas hace que tu API sea predecible para quien la usa (tu front, por ejemplo).

Un CRUD completo

src/demos/node/02-express/api-rest.ts

```
import express from 'express'
import { ofertas as iniciales, type Oferta } from './datos.ts'

export const app = express()
app.use(express.json())

let ofertas: Oferta[] = structuredClone(iniciales)
let ultimoId = ofertas.length

const buscar = (id: string) => ofertas.find((o) => o.id ===
  Number(id))

// Listar, con filtro y paginación: GET /ofertas?
  modalidad=remoto&pagina=1&porPagina=2
app.get('/ofertas', (req, res) => {
  const { modalidad } = req.query
  const pagina = Number(req.query.pagina ?? 1)
  const porPagina = Number(req.query.porPagina ?? 10)
  const lista = modalidad ? ofertas.filter((o) => o.modalidad ===
    modalidad) : ofertas
  res.json({
```

```
    datos: lista.slice((pagina - 1) * porPagina, pagina * porPagina),
    total: lista.length,
    pagina,
    porPagina,
  })
})

// Leer una
app.get('/ofertas/:id', (req, res) => {
  const oferta = buscar(req.params.id)
  if (!oferta) return res.status(404).json({ error: 'Oferta no
    encontrada' })
  res.json(oferta)
})

// Crear: 201 Created y dónde quedó el recurso nuevo
app.post('/ofertas', (req, res) => {
  const nueva: Oferta = { ...req.body, id: ++ultimoId }
  ofertas.push(nueva)
  res.status(201).location(`/ofertas/${nueva.id}`).json(nueva)
})

// Reemplazar entera (PUT) o cambiar algunos campos (PATCH)
app.put('/ofertas/:id', (req, res) => {
  const oferta = buscar(req.params.id)
  if (!oferta) return res.status(404).json({ error: 'Oferta no
    encontrada' })
  const reemplazo: Oferta = { ...req.body, id: oferta.id }
  ofertas = ofertas.map((o) => (o.id === oferta.id ? reemplazo : o))
  res.json(reemplazo)
})

app.patch('/ofertas/:id', (req, res) => {
  const oferta = buscar(req.params.id)
  if (!oferta) return res.status(404).json({ error: 'Oferta no
    encontrada' })
  Object.assign(oferta, req.body, { id: oferta.id })
  res.json(oferta)
})

// Borrar: 204 No Content (no hay nada que devolver)
app.delete('/ofertas/:id', (req, res) => {
  if (!buscar(req.params.id)) return res.status(404).json({ error:
    'Oferta no encontrada' })
  ofertas = ofertas.filter((o) => o.id !== Number(req.params.id))
  res.status(204).end()
})
```

src/demos/node/02-express/datos.ts

```ts
// Datos de ejemplo en memoria (cada ejemplo arranca con estos mismos)
export type Modalidad = 'remoto' | 'hibrido' | 'presencial'

export type Oferta = {
  id: number
  titulo: string
  empresa: string
  modalidad: Modalidad
  salarioMax: number | null
}

export const ofertas: Oferta[] = [
  { id: 1, titulo: 'Frontend React Senior', empresa: 'Nubelia',
    modalidad: 'remoto', salarioMax: 4000 },
  { id: 2, titulo: 'Fullstack Node', empresa: 'Nubelia', modalidad:
    'remoto', salarioMax: 3000 },
  { id: 3, titulo: 'React Native Developer', empresa: 'Pagos Sur',
    modalidad: 'hibrido', salarioMax: 3200 },
  { id: 4, titulo: 'Frontend Junior', empresa: 'Mate Cloud',
    modalidad: 'presencial', salarioMax: 1600 },
  { id: 5, titulo: 'Senior Frontend Engineer', empresa: 'Delta Pay',
    modalidad: 'remoto', salarioMax: 6000 },
]
```

PEDIDOS A LA APP · Node 24.22.1 · Express 5.3.0

GET `/ofertas?modalidad=remoto&pagina=1&porPagina=2`

← 200 OK


```
{
  "datos": [
    {
      "id": 1,
      "titulo": "Frontend React Senior",
      "empresa": "Nubelia",
      "modalidad": "remoto",
      "salarioMax": 4000
    },
    {
      "id": 2,
      "titulo": "Fullstack Node",
      "empresa": "Nubelia",
      "modalidad": "remoto",
      "salarioMax": 3000
    }
  ],
  "total": 3,
  "pagina": 1,
  "porPagina": 2
}
```


GET `/ofertas/3`

← 200 OK


```
{
  "id": 3,
  "titulo": "React Native Developer",
  "empresa": "Pagos Sur",
  "modalidad": "hibrido",
  "salarioMax": 3200
}
```

POST `/ofertas`

```
{
  "titulo": "Backend Node",
  "empresa": "Delta Pay",
  "modalidad": "remoto",
  "salarioMax": 4500
}
```

← 201 Created

```
location: /ofertas/6
```

```
{
  "titulo": "Backend Node",
  "empresa": "Delta Pay",
  "modalidad": "remoto",
  "salarioMax": 4500,
  "id": 6
}
```

PATCH `/ofertas/3`

```
{
  "salarioMax": 3500
}
```

← 200 OK

```
{
  "id": 3,
  "titulo": "React Native Developer",
  "empresa": "Pagos Sur",
  "modalidad": "hibrido",
  "salarioMax": 3500
}
```

PUT /ofertas/4

```
{
  "titulo": "Frontend Semi",
  "empresa": "Mate Cloud",
  "modalidad": "hibrido",
  "salarioMax": 2400
}
```

← 200 OK

```
{
  "titulo": "Frontend Semi",
  "empresa": "Mate Cloud",
  "modalidad": "hibrido",
  "salarioMax": 2400,
  "id": 4
}
```

DELETE /ofertas/2

← 204 No Content

DELETE /ofertas/2

← 404 Not Found

```
{
  "error": "Oferta no encontrada"
}
```

GET /ofertas?porPagina=3

← 200 OK


```
{
  "datos": [
    {
      "id": 1,
      "titulo": "Frontend React Senior",
      "empresa": "Nubelia",
      "modalidad": "remoto",
      "salarioMax": 4000
    },
    {
      "id": 3,
      "titulo": "React Native Developer",
      "empresa": "Pagos Sur",
      "modalidad": "hibrido",
      "salarioMax": 3500
    },
    {
      "titulo": "Frontend Semi",
      "empresa": "Mate Cloud",
      "modalidad": "hibrido",
      "salarioMax": 2400,
      "id": 4
    }
  ],
  "total": 5,
  "pagina": 1,
  "porPagina": 3
}
```


Las convenciones

Acción	Método y ruta	Respuesta
Listar	`GET /ofertas`	200 con la lista (paginada)
Leer una	`GET /ofertas/:id`	200, o 404 si no existe
Crear	`POST /ofertas`	**201** con el recurso creado y el header `Location`
Reemplazar	`PUT /ofertas/:id`	200 (o 204)
Modificar una parte	`PATCH /ofertas/:id`	200 con el recurso actualizado

Acción	Método y ruta	Respuesta
Borrar	`DELETE /ofertas/:id`	**204** sin cuerpo, o 404
Relaciones	`GET /empresas/:id/ofertas`	Las ofertas de una empresa
Acciones que no son CRUD	`POST /postulaciones/:id/archivar`	Un verbo como sub-recurso, con `POST`

- **Sustantivos en plural**, no verbos: `GET /ofertas`, no `GET /obtenerOfertas`. La acción ya la dice el método.
- **Filtros, orden y paginación en la query**: `?modalidad=remoto&orden=-salarioMax&pagina=2`.
- **Status codes con sentido**: el cliente decide qué hacer según el código, no según un `{ ok: false }` adentro de un 200.
- **Un formato de error consistente** en toda la API, por ejemplo `{ "error": "mensaje", "detalles": [...] }`.
- **Versionado**: cuando un cambio rompe a los clientes, una versión nueva (`/api/v2/ofertas`) en lugar de cambiar la existente.

Paginación

Una lista sin límite es una bomba de tiempo: hoy hay 15 ofertas, en un año 200.000 y el endpoint tarda 20 segundos. Dos formas:

	Por página (*offset*)	Por cursor
Pedido	`?pagina=3&porPagina=20`	`?despuesDe=1042&limite=20`
SQL	`LIMIT 20 OFFSET 40`	`WHERE id > 1042 ORDER BY id LIMIT 20`
A favor	Simple, permite saltar a la página 7 y mostrar el total	Igual de rápido en cualquier página; no se saltean ni repiten elementos si se agregan nuevos
Para	Tablas de un panel de administración	Feeds y scroll infinito

La respuesta suele incluir los datos y la información para pedir más: `{ datos, total, pagina, porPagina }` o `{ datos, siguienteCursor }`.

Detalles que suman

- **PUT vs PATCH**: con `PUT` mandás el recurso **entero** (lo que no mandás, se pierde); con `PATCH`, solo lo que cambia. En el ejemplo, el `PATCH` cambió el salario y dejó el resto.
- **Idempotencia**: el segundo `DELETE /ofertas/2` dio 404, pero el estado es el mismo (la oferta no está). Por eso un cliente puede reintentar un `DELETE` o un `PUT` sin miedo. Un `POST` repetido crea dos recursos: para pagos y cosas sensibles se usa un header `Idempotency-Key`.
- **No devuelvas de más**: nunca el hash de la contraseña, ni datos internos. Elegí los campos de la respuesta a propósito.
- **Documentación**: una API se documenta con **OpenAPI** (antes Swagger). Hay librerías que la generan desde los esquemas de Zod.

> ℹ️ **¿Y GraphQL?**
>
> **GraphQL** es la alternativa más conocida: un solo endpoint donde el cliente pide exactamente los campos que quiere, útil cuando hay muchos clientes con necesidades distintas. Agrega complejidad (caché, permisos por campo, N+1). Para la mayoría de las apps, una API REST bien hecha alcanza. Otra opción si el front y el back son TypeScript: **tRPC**, que comparte los tipos sin escribir una API "pública".

🎤 En una entrevista

"¿Qué hace que una API sea REST?" → "Que modela los datos como recursos con URLs (sustantivos en plural, como /ofertas/3) y usa los métodos HTTP para las acciones: GET para leer, POST para crear, PUT y PATCH para modificar, DELETE para borrar. Además, es sin estado (cada pedido trae todo lo necesario, por ejemplo el token) y usa los status codes con su significado: 201 al crear, 204 sin contenido, 400 si los datos son inválidos, 404 si no existe. Le agrego paginación, un formato de error consistente y versionado."

Capítulo 11

Validar con Zod

Body, query y params validados, errores 400 con detalle y mass assignment.

Todo lo que llega en un pedido lo escribió **alguien de afuera**: tu front, pero también cualquiera con `curl`. TypeScript no ayuda acá: `req.body` es `any`, y aunque le pongas un tipo, en ejecución puede venir cualquier cosa. Hay que **validar en el servidor**, siempre, aunque el front ya valide el formulario.

Zod es la librería más usada para eso: definís un esquema una vez y te da la validación en ejecución **y** el tipo de TypeScript. (La viste en la guía de TypeScript para validar respuestas de APIs; acá es el mismo uso, del otro lado.)

Validar body y query

src/demos/node/02-express/validacion.ts

```
import express, { type ErrorRequestHandler } from 'express'
import { z } from 'zod'
import { ofertas } from './datos.ts'

z.config(z.locales.es()) // mensajes de error en español

// El esquema describe y valida los datos; el tipo de TypeScript sale
  de ahí
const OfertaNueva = z.object({
  titulo: z.string().trim().min(3, 'El título necesita al menos 3
    letras'),
  empresa: z.string().trim().min(1),
  modalidad: z.enum(['remoto', 'hibrido',
    'presencial']).default('remoto'),
  salarioMax: z.number().int().positive().nullable().default(null),
```

```
})
type OfertaNueva = z.infer<typeof OfertaNueva> // { titulo: string;
  empresa: string; ... }

// La query siempre llega como texto: z.coerce la convierte a número
const Filtros = z.object({
  modalidad: z.enum(['remoto', 'hibrido', 'presencial']).optional(),
  minimo: z.coerce.number().int().nonnegative().default(0),
})

export const app = express()
app.use(express.json())

app.get('/ofertas', (req, res) => {
  const { modalidad, minimo } = Filtros.parse(req.query) // si no es
    válida, lanza un ZodError
  res.json(
    ofertas.filter((o) => (!modalidad || o.modalidad === modalidad) &&
      (o.salarioMax ?? 0) >= minimo),
  )
})

app.post('/ofertas', (req, res) => {
  const datos: OfertaNueva = OfertaNueva.parse(req.body)
  const nueva = { id: ofertas.length + 1, ...datos }
  ofertas.push(nueva)
  res.status(201).json(nueva)
})

// Un solo lugar convierte los errores de validación en un 400 con el
  detalle
const manejarErrores: ErrorRequestHandler = (error, _req, res, _next)
  => {
  if (error instanceof z.ZodError) {
    return res.status(400).json({
      error: 'Datos inválidos',
      detalles: error.issues.map((i) => ({ campo: i.path.join('.'),
        mensaje: i.message })),
    })
  }
  console.error(error)
  res.status(500).json({ error: 'Error interno' })
}
app.use(manejarErrores)
```

src/demos/node/02-express/datos.ts

```
// Datos de ejemplo en memoria (cada ejemplo arranca con estos mismos)
export type Modalidad = 'remoto' | 'hibrido' | 'presencial'
```

```ts
export type Oferta = {
  id: number
  titulo: string
  empresa: string
  modalidad: Modalidad
  salarioMax: number | null
}

export const ofertas: Oferta[] = [
  { id: 1, titulo: 'Frontend React Senior', empresa: 'Nubelia',
    modalidad: 'remoto', salarioMax: 4000 },
  { id: 2, titulo: 'Fullstack Node', empresa: 'Nubelia', modalidad:
    'remoto', salarioMax: 3000 },
  { id: 3, titulo: 'React Native Developer', empresa: 'Pagos Sur',
    modalidad: 'hibrido', salarioMax: 3200 },
  { id: 4, titulo: 'Frontend Junior', empresa: 'Mate Cloud',
    modalidad: 'presencial', salarioMax: 1600 },
  { id: 5, titulo: 'Senior Frontend Engineer', empresa: 'Delta Pay',
    modalidad: 'remoto', salarioMax: 6000 },
]
```

PEDIDOS A LA APP · Node 24.22.1 · Express 5.3.0

POST `/ofertas`

```
{
  "titulo": "Backend Node",
  "empresa": "Delta Pay",
  "salarioMax": 4500
}
```

← 201 Created

```
{
  "id": 6,
  "titulo": "Backend Node",
  "empresa": "Delta Pay",
  "modalidad": "remoto",
  "salarioMax": 4500
}
```

POST `/ofertas`

```
{
  "titulo": "QA",
  "modalidad": "en la luna",
  "salarioMax": -10
}
```

← 400 Bad Request

```
{
  "error": "Datos inválidos",
  "detalles": [
    {
      "campo": "titulo",
      "mensaje": "El título necesita al menos 3 letras"
    },
    {
      "campo": "empresa",
      "mensaje": "Entrada inválida: se esperaba texto, recibido
indefinido"
    },
    {
      "campo": "modalidad",
      "mensaje": "Opción inválida: se esperaba una de
\"remoto\"|\"hibrido\"|\"presencial\""
    },
    {
      "campo": "salarioMax",
      "mensaje": "Demasiado pequeño: se esperaba que número fuera >0"
    }
  ]
}
```

POST `/ofertas`

```
{
  "titulo": "  Data Engineer  ",
  "empresa": "Cumbre Data",
  "esAdmin": true
}
```

← 201 Created

```
{
  "id": 7,
  "titulo": "Data Engineer",
  "empresa": "Cumbre Data",
  "modalidad": "remoto",
  "salarioMax": null
}
```

GET `/ofertas?modalidad=remoto&minimo=3500`

← 200 OK

```
[
  {
    "id": 1,
    "titulo": "Frontend React Senior",
    "empresa": "Nubelia",
    "modalidad": "remoto",
    "salarioMax": 4000
  },
  {
    "id": 5,
    "titulo": "Senior Frontend Engineer",
    "empresa": "Delta Pay",
    "modalidad": "remoto",
    "salarioMax": 6000
  },
  {
    "id": 6,
    "titulo": "Backend Node",
    "empresa": "Delta Pay",
    "modalidad": "remoto",
    "salarioMax": 4500
  }
]
```

GET /ofertas?minimo=mucho

← 400 Bad Request

```
{
  "error": "Datos inválidos",
  "detalles": [
    {
      "campo": "minimo",
      "mensaje": "Entrada inválida: se esperaba número, recibido NaN"
    }
  ]
}
```

- `Esquema.parse(datos)` devuelve los datos validados (y **limpios**: con los valores por defecto, el `trim` aplicado y **sin los campos que no están en el esquema**, como el `esAdmin` del tercer pedido). Si no son válidos, lanza un `ZodError`.
- `z.infer<typeof Esquema>` es el tipo de TypeScript. Un solo lugar define la forma de los datos.
- `z.coerce.number()`: la query y los params siempre llegan como texto; `coerce` los convierte antes de validar.
- **Un manejador de errores** convierte cualquier `ZodError` en un 400 con el detalle de cada campo. Las rutas no tienen que acordarse.

> ⚠️ **Mass assignment**
>
> Sin validación, `{ id, ...req.body }` copia **todo** lo que manden. Si el objeto termina en la base de datos, alguien puede mandar `"esAdmin": true`, `"empresaId": 7` o `"aprobada": true`. Zod, por defecto, **descarta** los campos que no están en el esquema (con `z.strictObject` hace que fallen).

safeParse y un middleware

`parse` lanza un error; `safeParse` devuelve un resultado que hay que mirar:

```
const resultado = OfertaNueva.safeParse(req.body)
if (!resultado.success) {
  return res.status(400).json({ error: 'Datos inválidos', detalles:
    resultado.error.issues })
}
const oferta = resultado.data // tipada
```

Otra forma común es un middleware que valida antes de la ruta:

```
function validarBody(esquema: z.ZodType): RequestHandler {
  return (req, res, next) => {
    const resultado = esquema.safeParse(req.body)
    if (!resultado.success) return res.status(400).json({ detalles:
      resultado.error.issues })
    req.body = resultado.data // el body limpio
    next()
  }
}

app.post('/ofertas', validarBody(OfertaNueva), crearOferta)
```

> ℹ️ **req.query en Express 5**
>
> En Express 5, `req.query` es de solo lectura: `req.query = datosValidados` tira un error. Para la query, validá adentro de la ruta (como en el ejemplo) o guardá el resultado en `res.locals`.

Esquemas que vas a escribir

```
z.string().trim().min(1).max(150)
z.email()                                  // también z.url(),
  z.uuid(), z.iso.date()
z.number().int().positive()
```

```
z.coerce.number().int().min(1).default(1)  // de la query
z.enum(['remoto', 'hibrido', 'presencial'])
z.boolean().default(false)
z.array(z.string()).min(1)
z.object({ ... }).partial()                // todo opcional: el
  esquema de un PATCH
OfertaNueva.extend({ id: z.number() })     // agregar campos
OfertaNueva.pick({ titulo: true })         // quedarse con algunos
z.object({ desde: z.iso.date(), hasta: z.iso.date() })
  .refine((r) => r.desde <= r.hasta, 'desde tiene que ser anterior a
    hasta')
```

Un mismo esquema puede vivir en un paquete compartido entre el front y el back (en un monorepo): el formulario de React y la API validan exactamente lo mismo.

> **🎤 En una entrevista**
>
> "Si el front ya valida el formulario, ¿hace falta validar en el back?" → "Sí, siempre. La validación del front es para la experiencia del usuario; la del back es la que protege los datos, porque cualquiera puede llamar a la API sin pasar por el front. En el servidor valido body, query y params con un esquema (Zod), descarto los campos que no espero para evitar mass assignment, y devuelvo un 400 con el detalle de qué campo falló. TypeScript no alcanza: los tipos desaparecen en ejecución."

Capítulo 12

Capas y base de datos

Rutas, servicio y repositorio, inyección de dependencias y el repositorio de MySQL.

Con tres rutas, todo en un archivo está bien. Con cuarenta, las rutas que mezclan HTTP, reglas de negocio y SQL se vuelven imposibles de testear y de cambiar. La organización más común en APIs de Node separa **capas**, cada una con un solo trabajo.

Las capas

```
Pedido HTTP
   │
   ▼
Rutas (o controladores)   Leen y validan el pedido, llaman al
  servicio, arman la respuesta.
   │                      Saben de HTTP; no saben de SQL.
   ▼
Servicio                  Las reglas del negocio ("no se puede
  publicar dos veces la misma oferta").
   │                      No sabe de HTTP ni de SQL: se puede usar
     desde una ruta, un script o un test.
   ▼
Repositorio               El acceso a los datos: SQL, un ORM, una API
  externa o memoria.
   │                      Es lo único que cambia si cambiás de base de
     datos.
   ▼
MySQL
```

El ejemplo completo

La misma API de ofertas, partida en capas. Mirá las pestañas: cada archivo es una capa.

src/demos/node/02-express/capas/app.ts

```ts
import express, { type ErrorRequestHandler } from 'express'
import { ZodError } from 'zod'
import { ErrorDeNegocio, type Oferta } from './ofertas.ts'
import { crearRepoEnMemoria, type RepositorioOfertas } from
  './repositorio.ts'
import { crearRutasOfertas } from './rutas.ts'
import { crearServicioOfertas } from './servicio.ts'

const ejemplo: Oferta[] = [
  { id: 1, titulo: 'Frontend React Senior', empresa: 'Nubelia',
    modalidad: 'remoto', salarioMax: 4000 },
  { id: 2, titulo: 'React Native Developer', empresa: 'Pagos Sur',
    modalidad: 'hibrido', salarioMax: 3200 },
]

// La app recibe sus dependencias: en producción, el repo de MySQL; en
  los tests, uno en memoria.
export function crearApp({ repo = crearRepoEnMemoria(ejemplo) }: {
  repo?: RepositorioOfertas } = {}) {
  const app = express()
  app.use(express.json())
  app.use('/ofertas', crearRutasOfertas(crearServicioOfertas(repo)))
  app.use(manejarErrores)
  return app
}

const manejarErrores: ErrorRequestHandler = (error, _req, res, _next)
  => {
  if (error instanceof ZodError) {
    return res.status(400).json({ error: 'Datos inválidos', detalles:
      error.issues.map((i) => i.path.join('.')) })
  }
  if (error instanceof ErrorDeNegocio) return
    res.status(error.status).json({ error: error.message })
  console.error(error)
  res.status(500).json({ error: 'Error interno' })
}
```

src/demos/node/02-express/capas/rutas.ts

```ts
import { Router } from 'express'
import { Filtros, OfertaNueva } from './ofertas.ts'
import type { ServicioOfertas } from './servicio.ts'

// Las rutas solo traducen HTTP: leen y validan el pedido, llaman al
  servicio y responden.
export function crearRutasOfertas(servicio: ServicioOfertas) {
```

```
  const rutas = Router()

  rutas.get('/', async (req, res) => {
    res.json(await servicio.listar(Filtros.parse(req.query)))
  })

  rutas.get('/:id', async (req, res) => {
    res.json(await servicio.obtener(Number(req.params.id)))
  })

  rutas.post('/', async (req, res) => {
    const oferta = await
      servicio.publicar(OfertaNueva.parse(req.body))
    res.status(201).location(`/ofertas/${oferta.id}`).json(oferta)
  })

  return rutas
}
```

src/demos/node/02-express/capas/servicio.ts

```
import { ErrorDeNegocio, type Filtros, type OfertaNueva } from
  './ofertas.ts'
import type { RepositorioOfertas } from './repositorio.ts'

// El servicio tiene las reglas del negocio. No sabe nada de HTTP (ni
  de req ni de res),
// así se puede usar desde una ruta, un script o un test.
export function crearServicioOfertas(repo: RepositorioOfertas) {
  return {
    listar: (filtros: Filtros) => repo.listar(filtros),

    async obtener(id: number) {
      const oferta = await repo.buscar(id)
      if (!oferta) throw new ErrorDeNegocio(404, `No existe la oferta
        ${id}`)
      return oferta
    },

    async publicar(datos: OfertaNueva) {
      const deLaEmpresa = await repo.listar({ empresa: datos.empresa
        })
      if (deLaEmpresa.some((o) => o.titulo.toLowerCase() ===
        datos.titulo.toLowerCase())) {
        throw new ErrorDeNegocio(409, `${datos.empresa} ya publicó
          "${datos.titulo}"`)
      }
      return repo.crear(datos)
    },
  }
```

```
}

export type ServicioOfertas = ReturnType<typeof crearServicioOfertas>
```

src/demos/node/02-express/capas/repositorio.ts

```
import type { Filtros, Oferta, OfertaNueva } from './ofertas.ts'

// El repositorio es lo único que sabe DÓNDE se guardan los datos.
// El servicio y las rutas usan esta interfaz y no se enteran si es
  memoria o MySQL.
export interface RepositorioOfertas {
  listar(filtros: Filtros): Promise<Oferta[]>
  buscar(id: number): Promise<Oferta | undefined>
  crear(datos: OfertaNueva): Promise<Oferta>
}

// En memoria: para desarrollar y para los tests
export function crearRepoEnMemoria(iniciales: Oferta[] = []):
  RepositorioOfertas {
  const ofertas = structuredClone(iniciales)
  return {
    async listar({ modalidad, empresa }) {
      return ofertas.filter((o) => (!modalidad || o.modalidad ===
        modalidad) && (!empresa || o.empresa === empresa))
    },
    async buscar(id) {
      return ofertas.find((o) => o.id === id)
    },
    async crear(datos) {
      const oferta = { ...datos, id: Math.max(0, ...ofertas.map((o) =>
        o.id)) + 1 }
      ofertas.push(oferta)
      return oferta
    },
  }
}
```

src/demos/node/02-express/capas/ofertas.ts

```
import { z } from 'zod'

// El modelo: los tipos y las reglas de forma de una oferta, sin nada
  de HTTP ni de base de datos
export const Modalidad = z.enum(['remoto', 'hibrido', 'presencial'])

export const OfertaNueva = z.object({
  titulo: z.string().trim().min(3),
```

```
  empresa: z.string().trim().min(1),
  modalidad: Modalidad.default('remoto'),
  salarioMax: z.number().int().positive().nullable().default(null),
})
export type OfertaNueva = z.infer<typeof OfertaNueva>
export type Oferta = OfertaNueva & { id: number }

export const Filtros = z.object({
  modalidad: Modalidad.optional(),
  empresa: z.string().optional(),
})
export type Filtros = z.infer<typeof Filtros>

/** Un error esperable del negocio, con el status HTTP que le
  corresponde. */
export class ErrorDeNegocio extends Error {
  readonly status: number
  constructor(status: number, mensaje: string) {
    super(mensaje)
    this.status = status
  }
}
```

PEDIDOS A LA APP · Node 24.22.1 · Express 5.3.0

GET `/ofertas`

← 200 OK


```
[
  {
    "id": 1,
    "titulo": "Frontend React Senior",
    "empresa": "Nubelia",
    "modalidad": "remoto",
    "salarioMax": 4000
  },
  {
    "id": 2,
    "titulo": "React Native Developer",
    "empresa": "Pagos Sur",
    "modalidad": "hibrido",
    "salarioMax": 3200
  }
]
```

GET `/ofertas?modalidad=hibrido`

← 200 OK

```
[
  {
    "id": 2,
    "titulo": "React Native Developer",
    "empresa": "Pagos Sur",
    "modalidad": "hibrido",
    "salarioMax": 3200
  }
]
```

POST `/ofertas`

```
{
  "titulo": "Backend Node",
  "empresa": "Nubelia"
}
```

← 201 Created

```
location: /ofertas/3
```

```
{
  "titulo": "Backend Node",
  "empresa": "Nubelia",
  "modalidad": "remoto",
  "salarioMax": null,
  "id": 3
}
```

POST `/ofertas`

```
{
  "titulo": "frontend react senior",
  "empresa": "Nubelia"
}
```

← 409 Conflict

```
{
  "error": "Nubelia ya publicó \"frontend react senior\""
}
```

POST /ofertas

```
{
  "titulo": "QA"
}
```

← 400 Bad Request

```
{
  "error": "Datos inválidos",
  "detalles": [
    "titulo",
    "empresa"
  ]
}
```

GET /ofertas/7

← 404 Not Found

```
{
  "error": "No existe la oferta 7"
}
```

- `ofertas.ts`: el modelo. Los esquemas de Zod, los tipos que salen de ellos y `ErrorDeNegocio`, un error que ya sabe qué status HTTP le corresponde.
- `repositorio.ts`: una **interfaz** (`RepositorioOfertas`) y una implementación en memoria.
- `servicio.ts`: la regla del título repetido (409) y el 404. Recibe el repositorio como parámetro.
- `rutas.ts`: valida con Zod y llama al servicio. Ninguna línea de SQL ni de reglas.
- `app.ts`: **arma las piezas**. `crearApp` recibe el repositorio, y por defecto usa uno en memoria.

Inyección de dependencias, sin frameworks

Que el servicio **reciba** el repositorio (en lugar de importarlo directo) es **inyección de dependencias**. Con funciones que reciben sus

dependencias alcanza; no hace falta un framework. La ganancia: la misma app funciona con datos en memoria en los tests y con MySQL en producción.

src/demos/node/02-express/capas/repositorio-mysql.ts

```ts
import type { Pool, ResultSetHeader, RowDataPacket } from
  'mysql2/promise'
import type { Oferta } from './ofertas.ts'
import type { RepositorioOfertas } from './repositorio.ts'

type Fila = RowDataPacket & { id: number; titulo: string; empresa:
  string; modalidad: Oferta['modalidad']; salario_max: number | null }

const aOferta = (f: Fila): Oferta => ({
  id: f.id,
  titulo: f.titulo,
  empresa: f.empresa,
  modalidad: f.modalidad,
  salarioMax: f.salario_max,
})

// La misma interfaz, guardando en MySQL (las tablas de la guía de
  MySQL, simplificadas)
export function crearRepoMysql(pool: Pool): RepositorioOfertas {
  return {
    async listar({ modalidad, empresa }) {
      const [filas] = await pool.execute<Fila[]>(
        `SELECT o.id, o.titulo, e.nombre AS empresa, o.modalidad,
          o.salario_max
         FROM ofertas o JOIN empresas e ON e.id = o.empresa_id
         WHERE (? IS NULL OR o.modalidad = ?) AND (? IS NULL OR
           e.nombre = ?)
         ORDER BY o.id`,
        [modalidad ?? null, modalidad ?? null, empresa ?? null,
          empresa ?? null],
      )
      return filas.map(aOferta)
    },
    async buscar(id) {
      const [filas] = await pool.execute<Fila[]>(
        `SELECT o.id, o.titulo, e.nombre AS empresa, o.modalidad,
          o.salario_max
         FROM ofertas o JOIN empresas e ON e.id = o.empresa_id
         WHERE o.id = ?`,
        [id],
      )
      return filas[0] && aOferta(filas[0])
```

```
    },
    async crear(datos) {
      const [resultado] = await pool.execute<ResultSetHeader>(
        `INSERT INTO ofertas (empresa_id, titulo, modalidad,
          salario_max, seniority, publicada)
         SELECT id, ?, ?, ?, 'semi', CURDATE() FROM empresas WHERE
           nombre = ?`,
        [datos.titulo, datos.modalidad, datos.salarioMax,
          datos.empresa],
      )
      return { ...datos, id: resultado.insertId }
    },
  }
}
```

src/demos/node/02-express/capas/servidor.ts

```
import mysql from 'mysql2/promise'
import { crearApp } from './app.ts'
import { crearRepoMysql } from './repositorio-mysql.ts'

// El punto de entrada en producción: arma las piezas de verdad y
  levanta el servidor.
const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
})

const app = crearApp({ repo: crearRepoMysql(pool) })
const puerto = Number(process.env.PUERTO ?? 3000)
app.listen(puerto, () => console.log(`API en
  http://localhost:${puerto}`))
```

El repositorio de MySQL cumple la misma interfaz con consultas con parámetros (como en el capítulo de MySQL desde Node). `servidor.ts` es el punto de entrada de producción: crea el pool, arma la app con el repositorio de verdad y la levanta. Para cambiar de base de datos, se cambia un archivo.

💡 **Transacciones entre repositorios**

Si una operación del servicio toca varias tablas (crear un pedido y descontar el stock), tiene que ir en **una transacción**. Una forma simple: que el repositorio reciba la conexión como parámetro, y que el servicio abra la transacción con `pool.getConnection()` y se la pase a cada repositorio. Los ORMs lo resuelven con algo como `prisma.$transaction(async (tx) => { ... })`.

¿Hace falta tanto?

Para un CRUD chico, rutas + un módulo de acceso a datos alcanza. Las capas valen la pena cuando hay **reglas de negocio** de verdad, varias fuentes de datos, o cuando querés testear la lógica sin levantar una base. Lo que no conviene es el extremo opuesto: SQL adentro de las rutas de Express, repetido en cinco lugares.

Otras arquitecturas que vas a escuchar: **MVC** (modelo, vista, controlador: el "controlador" es lo que acá llamamos rutas), **hexagonal** o *ports and adapters* (la misma idea de las interfaces llevada al extremo), y la organización **por feature** (una carpeta `ofertas/` con sus rutas, servicio y repositorio) en lugar de por tipo (una carpeta `rutas/`, otra `servicios/`...). Por feature escala mejor.

🎤 **En una entrevista**

"¿Cómo estructurarías una API en Node?" → "Por feature (una carpeta por recurso) y en capas: rutas que solo traducen HTTP y validan con Zod, un servicio con las reglas de negocio que no sabe nada de HTTP, y un repositorio que es lo único que habla con la base. Las dependencias se pasan por parámetro (el servicio recibe el repositorio), así en los tests uso uno en memoria y en producción el de MySQL. Los errores esperables los lanza el servicio con su status y los traduce un único middleware de errores."

Capítulo 13

Autenticación y seguridad

Contraseñas con hash, sesiones o JWT, 401 y 403, y la seguridad básica de una API.

Autenticación es saber quién es el usuario (login). **Autorización** es decidir qué puede hacer (un admin sí, un usuario no). Son las partes donde un error sale más caro, así que la regla número uno es: usá librerías probadas, no inventes tu propia criptografía.

Guardar contraseñas

Una contraseña **nunca** se guarda tal cual (ni cifrada, que se puede descifrar). Se guarda un **hash lento con sal**:

src/demos/node/02-express/auth/claves.ts

```
import { randomBytes, scrypt, timingSafeEqual } from 'node:crypto'
import { promisify } from 'node:util'

const scryptAsync = promisify(scrypt) as (clave: string, sal: Buffer,
  largo: number) => Promise<Buffer>

// Nunca se guarda la contraseña: se guarda un hash lento con una
  "sal" al azar.
export async function hashear(clave: string) {
  const sal = randomBytes(16)
  const hash = await scryptAsync(clave, sal, 64)
  return `${sal.toString('hex')}:${hash.toString('hex')}`
}

export async function verificar(clave: string, guardado: string) {
  const [sal, hash] = guardado.split(':')
  const calculado = await scryptAsync(clave, Buffer.from(sal, 'hex'),
    64)
```

```
  // timingSafeEqual tarda lo mismo aunque difieran en el primer byte
    (no da pistas)
  return timingSafeEqual(calculado, Buffer.from(hash, 'hex'))
}

if (import.meta.main) {
  const guardado = await hashear('secreto-123')
  console.log('Lo que se guarda:', guardado.length, 'caracteres (sal +
    hash en hexadecimal)')
  console.log('¿La misma clave da el mismo hash?', guardado === (await
    hashear('secreto-123')))
  console.log('Clave correcta:', await verificar('secreto-123',
    guardado))
  console.log('Clave incorrecta:', await verificar('secreto-124',
    guardado))
}
```

$ node claves.ts · Node 24.22.1

```
Lo que se guarda: 161 caracteres (sal + hash en hexadecimal)
¿La misma clave da el mismo hash? false
Clave correcta: true
Clave incorrecta: false
```

- **Hash**: una función de un solo camino. De la contraseña sale el hash, pero del hash no se puede volver a la contraseña. Para verificar un login, se calcula el hash de lo que escribió el usuario y se compara.
- **Lento a propósito**: con un hash rápido (como SHA-256), alguien que roba la base prueba miles de millones de contraseñas por segundo. `scrypt`, `bcrypt` y `argon2` tardan decenas de milisegundos cada vez.
- **Sal**: bytes al azar, distintos para cada usuario, guardados junto al hash. Dos usuarios con la misma contraseña tienen hashes distintos, y las tablas precalculadas no sirven.

El ejemplo usa `scrypt`, que viene con Node. En proyectos vas a ver **bcrypt** (el más común) o **argon2** (el recomendado hoy). Los tres están bien.

Sesiones o tokens

Después del login, el servidor tiene que reconocer al usuario en cada pedido siguiente. Dos formas:

	Sesión con cookie	Token (JWT)
Cómo funciona	El servidor guarda la sesión (en memoria, Redis o la base) y le da al navegador un id en una cookie	El servidor firma un token con los datos del usuario y el cliente lo manda en cada pedido
El servidor guarda	Las sesiones	Nada (solo el secreto para firmar)
Cerrar sesión / revocar	Borrar la sesión: inmediato	Difícil: el token vale hasta que vence (por eso duran poco)
Ideal para	Una web con su propio backend	APIs para apps móviles, o entre servicios

Las dos son válidas. Lo que **no** conviene es guardar un token en `localStorage`: cualquier script que logre ejecutarse en tu página (un XSS) lo puede leer. Para una web, una cookie `httpOnly` (que JavaScript no puede leer), `Secure` y `SameSite`, tenga adentro un id de sesión o el JWT.

JWT con Express

Un **JWT** son tres partes en base64 separadas por puntos: `header.payload.firma`. El payload **no está cifrado**, cualquiera lo puede leer: no pongas datos sensibles. La firma garantiza que nadie lo modificó, porque para firmar hace falta el secreto del servidor.

`src/demos/node/02-express/auth/app.ts`

```
import express, { type RequestHandler } from 'express'
import { jwtVerify, SignJWT } from 'jose'
import { hashear, verificar } from './claves.ts'

// En producción, el secreto sale de una variable de entorno larga y
  aleatoria
```

```
const secreto = new TextEncoder().encode(process.env.JWT_SECRET ??
  'secreto-solo-para-desarrollo')

type Usuario = { id: string; email: string; rol: 'admin' | 'usuario';
  hash: string }
const usuarios: Usuario[] = [
  { id: 'u1', email: 'ana@ejemplo.com', rol: 'usuario', hash: await
    hashear('secreto-123') },
]

export const app = express()
app.use(express.json())

// Login: si la clave es correcta, devuelve un token firmado que vence
  en 15 minutos
app.post('/login', async (req, res) => {
  const usuario = usuarios.find((u) => u.email === req.body.email)
  if (!usuario || !(await verificar(req.body.password, usuario.hash)))
    {
    return res.status(401).json({ error: 'Email o contraseña
      incorrectos' }) // no decir cuál falló
  }
  const token = await new SignJWT({ rol: usuario.rol })
    .setProtectedHeader({ alg: 'HS256' })
    .setSubject(usuario.id)
    .setIssuedAt()
    .setExpirationTime('15m')
    .sign(secreto)
  res.json({ token })
})

// Middleware: verifica el token del header Authorization y deja el
  usuario en res.locals
const autenticar: RequestHandler = async (req, res, next) => {
  const token = req.get('authorization')?.replace(/^Bearer /, '')
  if (!token) return res.status(401).json({ error: 'Falta el token' })
  try {
    const { payload } = await jwtVerify(token, secreto)
    res.locals.usuario = { id: payload.sub, rol: payload.rol }
    next()
  } catch {
    res.status(401).json({ error: 'Token inválido o vencido' })
  }
}

const soloAdmin: RequestHandler = (_req, res, next) => {
  if (res.locals.usuario.rol !== 'admin') return
    res.status(403).json({ error: 'Solo para administradores' })
  next()
}
```

```
app.get('/yo', autenticar, (_req, res) => {
  res.json(res.locals.usuario)
})

app.get('/admin/usuarios', autenticar, soloAdmin, (_req, res) => {
  res.json(usuarios.map(({ id, email }) => ({ id, email })))
})
```

src/demos/node/02-express/auth/claves.ts

```
import { randomBytes, scrypt, timingSafeEqual } from 'node:crypto'
import { promisify } from 'node:util'

const scryptAsync = promisify(scrypt) as (clave: string, sal: Buffer,
  largo: number) => Promise<Buffer>

// Nunca se guarda la contraseña: se guarda un hash lento con una
  "sal" al azar.
export async function hashear(clave: string) {
  const sal = randomBytes(16)
  const hash = await scryptAsync(clave, sal, 64)
  return `${sal.toString('hex')}:${hash.toString('hex')}`
}

export async function verificar(clave: string, guardado: string) {
  const [sal, hash] = guardado.split(':')
  const calculado = await scryptAsync(clave, Buffer.from(sal, 'hex'),
    64)
  // timingSafeEqual tarda lo mismo aunque difieran en el primer byte
    (no da pistas)
  return timingSafeEqual(calculado, Buffer.from(hash, 'hex'))
}

if (import.meta.main) {
  const guardado = await hashear('secreto-123')
  console.log('Lo que se guarda:', guardado.length, 'caracteres (sal +
    hash en hexadecimal)')
  console.log('¿La misma clave da el mismo hash?', guardado === (await
    hashear('secreto-123')))
  console.log('Clave correcta:', await verificar('secreto-123',
    guardado))
  console.log('Clave incorrecta:', await verificar('secreto-124',
    guardado))
}
```

PEDIDOS A LA APP · Node 24.22.1 · Express 5.3.0

GET `/yo`

← 401 Unauthorized

```
{
  "error": "Falta el token"
}
```

POST `/login`

```
{
  "email": "ana@ejemplo.com",
  "password": "incorrecta"
}
```

← 401 Unauthorized

```
{
  "error": "Email o contraseña incorrectos"
}
```

POST `/login`

```
{
  "email": "ana@ejemplo.com",
  "password": "secreto-123"
}
```

← 200 OK

```
{
  "token":
"eyJhbGciOiJIUzI1NiJ9.eyJyb2wiOiJ1c3VhcmlvIiwic3ViIjoidTEiLCJpYXQiOjE
3OTMzNDI0MDUsImV4cCI6MTc5MzM0MzMwNX0.XYHnyDaDzeeyw5TyqnfIgj7Tn1CwbRl-
nzFYhdxzwZs"
}
```

GET `/yo` authorization: Bearer {token}

← 200 OK

```
{
  "id": "u1",
  "rol": "usuario"
}
```

GET /admin/usuarios authorization: Bearer {token}

← 403 Forbidden

```
{
  "error": "Solo para administradores"
}
```

GET /yo authorization: Bearer token-inventado

← 401 Unauthorized

```
{
  "error": "Token inválido o vencido"
}
```

(`{token}` es el token que devolvió el login: la guía lo reemplaza al hacer el pedido.)

- El login responde lo mismo si el email no existe o si la contraseña está mal: así no se puede averiguar qué emails están registrados.
- `autenticar` verifica la firma y el vencimiento, y deja el usuario en `res.locals`. Sin token o con uno inválido: **401**.
- `soloAdmin` chequea el rol: Ana está autenticada pero no es admin, entonces **403**.
- El token vence en 15 minutos. Para no pedir la contraseña cada 15 minutos se usa un **refresh token** (de larga duración, en una cookie `httpOnly`, guardado en la base para poder revocarlo) que pide tokens nuevos.

El ejemplo usa **jose**, que funciona en Node, en el navegador y en el edge. En proyectos más viejos vas a ver **jsonwebtoken** (`jwt.sign` y `jwt.verify`), que hace lo mismo. Y para no hacer todo esto a mano existen **Auth.js**, **Better Auth**, **Passport** o servicios como **Clerk**, **Auth0** o **Supabase Auth**.

Seguridad básica de una API

Riesgo	Qué hacer
Inyección SQL	Consultas con parámetros, siempre (MySQL desde Node)
Datos inválidos o de más	Validar todo con Zod (validación)
Fuerza bruta en el login	`express-rate-limit`: por ejemplo, 5 intentos por minuto por IP
Headers inseguros	`helmet()`: agrega headers como `Content-Security-Policy` y `Strict-Transport-Security`, y saca `X-Powered-By`
Secretos en el código	Variables de entorno; el `.env` fuera del repo
CORS demasiado abierto	Lista de orígenes permitidos (middlewares)
Errores que muestran de más	En producción, al cliente un mensaje genérico; el detalle (stack, SQL) solo en los logs
Dependencias con vulnerabilidades	`npm audit`, Dependabot o Renovate
Acceso a datos ajenos	Chequear que el recurso **es del usuario**: `GET /postulaciones/7` no alcanza con estar logueado, la postulación 7 tiene que ser tuya

La última es de las más comunes en la vida real (se llama *IDOR*, o *broken object level authorization*): el usuario está autenticado, pero nadie verifica que el objeto que pide le pertenezca.

```
import helmet from 'helmet'
import rateLimit from 'express-rate-limit'

app.use(helmet())
app.use('/login', rateLimit({ windowMs: 60_000, limit: 5 }))
```

🎤 En una entrevista

"¿Cómo implementarías la autenticación de una API?" → "Las contraseñas se guardan con un hash lento y con sal (bcrypt, argon2 o scrypt). En el login verifico el hash y emito una sesión o un JWT de vida corta firmado con un secreto de las variables de entorno. Un middleware lo verifica en cada pedido y deja el usuario disponible: 401 si no hay token válido, 403 si el usuario no tiene permiso. En una web prefiero cookies httpOnly, Secure y SameSite en lugar de localStorage, por XSS. Sumo rate limiting en el login, helmet, CORS con una lista de orígenes, y en cada recurso verifico que pertenezca al usuario."

Capítulo 14

Tests con node:test

Unit tests, mocks y tests de la API entera, sin instalar nada.

Node trae su propio **test runner** (`node:test`) y una librería de aserciones (`node:assert`). Para una API alcanzan: no hace falta instalar Jest ni Vitest (aunque los vas a encontrar en muchos proyectos, y se escriben casi igual).

Tests unitarios

Una función, sus casos y sus bordes:

src/demos/node/02-express/testing/precios.test.ts

```
import assert from 'node:assert/strict'
import { describe, test } from 'node:test'
import { rangoSalarial } from './precios.ts'

describe('rangoSalarial', () => {
  test('con mínimo y máximo', () => {
    assert.equal(rangoSalarial(3000, 4000), 'USD 3000 - 4000')
  })

  test('sin datos', () => {
    assert.equal(rangoSalarial(null, null), 'A convenir')
  })

  test('solo uno de los dos', () => {
    assert.equal(rangoSalarial(null, 4000), 'Hasta USD 4000')
    assert.equal(rangoSalarial(3000, null), 'Desde USD 3000')
  })

  test('mínimo mayor que máximo tira un error', () => {
    assert.throws(() => rangoSalarial(5000, 4000), RangeError)
```

```
  })
})
```

src/demos/node/02-express/testing/precios.ts

```ts
export function rangoSalarial(min: number | null, max: number | null)
  {
  if (min === null && max === null) return 'A convenir'
  if (min === null) return `Hasta USD ${max}`
  if (max === null) return `Desde USD ${min}`
  if (min > max) throw new RangeError('El mínimo no puede ser mayor
    que el máximo')
  return `USD ${min} - ${max}`
}
```

$ node --test precios.test.ts · Node 24.22.1

```
▶ rangoSalarial
  ✔ con mínimo y máximo (…ms)
  ✔ sin datos (…ms)
  ✔ solo uno de los dos (…ms)
  ✔ mínimo mayor que máximo tira un error (…ms)
✔ rangoSalarial (…ms)
ℹ tests 4
ℹ suites 1
ℹ pass 4
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms …
```

- `node --test` busca y ejecuta los archivos de test (los que terminan en `.test.ts`, `.test.js`, o están en una carpeta `test/`). Con un archivo, ejecuta ese.
- `describe` agrupa y `test` (o `it`) define un caso.
- `assert/strict`: `equal` (===), `deepEqual` (compara objetos y arrays por contenido), `throws` y `rejects` (que lance un error, sincrónico o async), `match` (contra una regex).
- `node --test --watch` vuelve a correr al guardar, y `--experimental-test-coverage` muestra la cobertura.

Tests con mocks

Para testear el servicio **sin base de datos**, se le pasa un repositorio falso. Gracias a la inyección de dependencias, no hace falta ningún truco:

src/demos/node/02-express/testing/servicio.test.ts

```ts
import assert from 'node:assert/strict'
import { mock, test } from 'node:test'
import type { RepositorioOfertas } from '../capas/repositorio.ts'
import { crearServicioOfertas } from '../capas/servicio.ts'

// Un repositorio falso: cada método es un mock que registra cómo lo
  llamaron
function repoFalso(existentes:
  Awaited<ReturnType<RepositorioOfertas['listar']>> = []) {
  return {
    listar: mock.fn(async () => existentes),
    buscar: mock.fn(async () => undefined),
    crear: mock.fn(async (datos) => ({ ...datos, id: 99 })),
  } satisfies RepositorioOfertas
}

const nueva = { titulo: 'Backend Node', empresa: 'Nubelia', modalidad:
  'remoto' as const, salarioMax: null }

test('publicar guarda la oferta si no está repetida', async () => {
  const repo = repoFalso()
  const oferta = await crearServicioOfertas(repo).publicar(nueva)
  assert.equal(oferta.id, 99)
  assert.equal(repo.crear.mock.callCount(), 1)
  assert.deepEqual(repo.listar.mock.calls[0].arguments, [{ empresa:
    'Nubelia' }])
})

test('publicar rechaza un título repetido en la misma empresa (409)',
  async () => {
  const repo = repoFalso([{ ...nueva, titulo: 'backend node', id: 1
    }])
  await assert.rejects(crearServicioOfertas(repo).publicar(nueva), {
    status: 409 })
  assert.equal(repo.crear.mock.callCount(), 0)
})

test('obtener una que no existe da 404', async () => {
  await assert.rejects(crearServicioOfertas(repoFalso()).obtener(7), {
    status: 404 })
```

```
})
```

src/demos/node/02-express/capas/servicio.ts

```ts
import { ErrorDeNegocio, type Filtros, type OfertaNueva } from
  './ofertas.ts'
import type { RepositorioOfertas } from './repositorio.ts'

// El servicio tiene las reglas del negocio. No sabe nada de HTTP (ni
  de req ni de res),
// así se puede usar desde una ruta, un script o un test.
export function crearServicioOfertas(repo: RepositorioOfertas) {
  return {
    listar: (filtros: Filtros) => repo.listar(filtros),

    async obtener(id: number) {
      const oferta = await repo.buscar(id)
      if (!oferta) throw new ErrorDeNegocio(404, `No existe la oferta
        ${id}`)
      return oferta
    },

    async publicar(datos: OfertaNueva) {
      const deLaEmpresa = await repo.listar({ empresa: datos.empresa
        })
      if (deLaEmpresa.some((o) => o.titulo.toLowerCase() ===
        datos.titulo.toLowerCase())) {
        throw new ErrorDeNegocio(409, `${datos.empresa} ya publicó
          "${datos.titulo}"`)
      }
      return repo.crear(datos)
    },
  }
}

export type ServicioOfertas = ReturnType<typeof crearServicioOfertas>
```

$ node --test servicio.test.ts · Node 24.22.1

```
✔ publicar guarda la oferta si no está repetida (…ms)
✔ publicar rechaza un título repetido en la misma empresa (409) (…ms)
✔ obtener una que no existe da 404 (…ms)
ℹ tests 3
ℹ suites 0
ℹ pass 3
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
```

```
ℹ todo 0
ℹ duration_ms …
```

`mock.fn(implementación)` crea una función que registra cada llamada: `mock.callCount()` y `mock.calls[i].arguments` permiten verificar **cómo** se usó la dependencia (que se llamó a `crear`, o que **no** se llamó, en el caso del título repetido). Para reemplazar un método de un objeto existente está `t.mock.method(objeto, 'metodo')`, y para el reloj, `t.mock.timers`.

Tests de la API

Un test de integración levanta la app de verdad (en un puerto libre, con `listen(0)`) y le hace pedidos HTTP con `fetch`. Prueba juntas las rutas, la validación, el servicio y el manejo de errores:

src/demos/node/02-express/testing/app.test.ts

```ts
import assert from 'node:assert/strict'
import type { AddressInfo } from 'node:net'
import { after, before, describe, test } from 'node:test'
import { crearApp } from '../capas/app.ts'

// Tests de la API entera: se levanta en un puerto libre y se le pega
  con fetch
describe('API de ofertas', () => {
  let base = ''
  const servidor = crearApp().listen(0)
  before(() => {
    base = `http://localhost:${(servidor.address() as
      AddressInfo).port}`
  })
  after(() => servidor.close())

  const publicar = (cuerpo: object) =>
    fetch(`${base}/ofertas`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(cuerpo),
    })

  test('GET /ofertas lista las ofertas', async () => {
```

```
    const res = await fetch(`${base}/ofertas`)
    assert.equal(res.status, 200)
    const ofertas = (await res.json()) as unknown[]
    assert.equal(ofertas.length, 2)
  })

  test('POST /ofertas crea una oferta (201)', async () => {
    const res = await publicar({ titulo: 'Backend Node', empresa:
      'Nubelia' })
    assert.equal(res.status, 201)
    assert.equal(res.headers.get('location'), '/ofertas/3')
    const oferta = (await res.json()) as { modalidad: string }
    assert.equal(oferta.modalidad, 'remoto') // el valor por defecto
  })

  test('POST /ofertas con datos inválidos da 400', async () => {
    const res = await publicar({ titulo: 'X' })
    assert.equal(res.status, 400)
    const error = (await res.json()) as { detalles: string[] }
    assert.deepEqual(error.detalles, ['titulo', 'empresa'])
  })

  test('GET /ofertas/99 da 404', async () => {
    assert.equal((await fetch(`${base}/ofertas/99`)).status, 404)
  })
})
```

$ node --test app.test.ts · Node 24.22.1

```
▶ API de ofertas
  ✔ GET /ofertas lista las ofertas (…ms)
  ✔ POST /ofertas crea una oferta (201) (…ms)
  ✔ POST /ofertas con datos inválidos da 400 (…ms)
  ✔ GET /ofertas/99 da 404 (…ms)
✔ API de ofertas (…ms)
ℹ tests 4
ℹ suites 1
ℹ pass 4
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms …
```

- `before` y `after` preparan y limpian (levantar y cerrar el servidor). También hay `beforeEach` y `afterEach`.

- Como `crearApp()` usa un repositorio en memoria por defecto, cada archivo de test arranca con datos conocidos.
- En muchos proyectos vas a ver **supertest**, que hace lo mismo sin levantar el puerto a mano: `await request(app).post('/ofertas').send({...}).expect(201)`.

> **ℹ️ ¿Y la base de datos en los tests?**
>
> Los tests del repositorio de MySQL necesitan un MySQL de verdad. Lo habitual es levantar uno en Docker para los tests (a mano con `docker compose`, o con **Testcontainers**, que lo levanta desde el código), cargar el esquema, y limpiar las tablas entre tests. Son más lentos: se escriben menos y para lo que importa (las consultas complicadas).

Qué testear

- **Mucho** test unitario de la lógica de negocio (los servicios): rápidos y precisos.
- **Algunos** tests de la API (integración): los caminos principales y los errores (400, 401, 404, 409).
- **Pocos** tests de punta a punta con base de datos y todo.

Y lo de siempre: un test que nunca falla no prueba nada. Mirá que falle cuando rompés el código.

🎤 En una entrevista

"¿Cómo testearías una API de Express?" → "Separo la app del listen para poder importarla. La lógica de negocio va en servicios que reciben sus dependencias, y los testeo con unit tests y repositorios falsos o mocks. Después, tests de integración que levantan la app (con supertest o con listen(0) y fetch) y verifican status, body y headers de los casos principales y de error. Para la base, unos pocos tests contra un MySQL en Docker. Uso node:test o Vitest, y todo corre en CI en cada PR."

Capítulo 15

A producción

Configuración validada, logs, errores, apagado prolijo, health check y Docker.

Lo que separa una API que funciona en tu máquina de una que funciona en producción: configuración, logs, errores, apagado y deploy.

Configuración validada al arrancar

src/demos/node/02-express/produccion/config.ts

```
import { z } from 'zod'

z.config(z.locales.es())

// La configuración se valida UNA vez, al arrancar. Si falta algo, la
  app no arranca:
// mejor un error claro en el deploy que un undefined a las 3 de la
  mañana.
const Entorno = z.object({
  NODE_ENV: z.enum(['development', 'test',
    'production']).default('development'),
  PUERTO: z.coerce.number().int().default(3000),
  DB_HOST: z.string().min(1),
  DB_USER: z.string().min(1),
  DB_PASSWORD: z.string().min(1),
  DB_NAME: z.string().min(1),
  JWT_SECRET: z.string().min(32, 'Tiene que tener al menos 32
    caracteres'),
})

const resultado = Entorno.safeParse(process.env)
if (!resultado.success) {
  console.error('Configuración inválida:')
```

```
  for (const problema of resultado.error.issues) console.error(`
    ${problema.path.join('.')}: ${problema.message}`)
  process.exit(1)
}

export const config = resultado.data // tipada: config.PUERTO es
  number

if (import.meta.main) {
  console.log('Configuración OK:', { ...config, DB_PASSWORD: '***',
    JWT_SECRET: '***' })
}
```

$ node config.ts · Node 24.22.1

```
Configuración inválida:
  DB_HOST: Entrada inválida: se esperaba texto, recibido indefinido
  DB_USER: Entrada inválida: se esperaba texto, recibido indefinido
  DB_PASSWORD: Entrada inválida: se esperaba texto, recibido
indefinido
  DB_NAME: Entrada inválida: se esperaba texto, recibido indefinido
  JWT_SECRET: Entrada inválida: se esperaba texto, recibido
indefinido
```

El proceso terminó con código 1

src/demos/node/02-express/produccion/config.ts

```
import { z } from 'zod'

z.config(z.locales.es())

// La configuración se valida UNA vez, al arrancar. Si falta algo, la
  app no arranca:
// mejor un error claro en el deploy que un undefined a las 3 de la
  mañana.
const Entorno = z.object({
  NODE_ENV: z.enum(['development', 'test',
    'production']).default('development'),
  PUERTO: z.coerce.number().int().default(3000),
  DB_HOST: z.string().min(1),
  DB_USER: z.string().min(1),
  DB_PASSWORD: z.string().min(1),
  DB_NAME: z.string().min(1),
  JWT_SECRET: z.string().min(32, 'Tiene que tener al menos 32
    caracteres'),
})
```

```ts
const resultado = Entorno.safeParse(process.env)
if (!resultado.success) {
  console.error('Configuración inválida:')
  for (const problema of resultado.error.issues) console.error(`
    ${problema.path.join('.')}: ${problema.message}`)
  process.exit(1)
}

export const config = resultado.data // tipada: config.PUERTO es
  number

if (import.meta.main) {
  console.log('Configuración OK:', { ...config, DB_PASSWORD: '***',
    JWT_SECRET: '***' })
}
```

`$ node --env-file=.env.ejemplo config.ts` · Node 24.22.1

```
Configuración OK: {
  NODE_ENV: 'production',
  PUERTO: 8080,
  DB_HOST: 'db.interna',
  DB_USER: 'app_busqueda',
  DB_PASSWORD: '***',
  DB_NAME: 'busqueda',
  JWT_SECRET: '***'
}
```

- Las variables se validan **todas juntas, al arrancar**. Si falta una, la app no arranca y el error dice cuál. Sin esto, la falta de `DB_PASSWORD` aparece como un error raro en el primer pedido, horas después del deploy.
- El resto del código importa `config` (tipado: `config.PUERTO` es `number`) en lugar de leer `process.env` por todos lados.
- Los secretos nunca se imprimen ni se loguean.

Logs

`console.log` sirve para empezar. En producción se usa un **logger estructurado** como **pino**: escribe cada línea como JSON (fácil de buscar y filtrar en el servicio de logs), con niveles y muy poco costo.

```
import pino from 'pino'
import pinoHttp from 'pino-http'

export const logger = pino({ level: process.env.LOG_LEVEL ?? 'info' })
app.use(pinoHttp({ logger })) // una línea por pedido: método, ruta,
  status y duración

logger.info({ ofertaId: 42 }, 'Oferta publicada')
logger.error({ err }, 'No se pudo guardar la postulación')
// {"level":30,"time":1793318400000,"ofertaId":42,"msg":"Oferta
  publicada"}
```

Niveles: `fatal`, `error`, `warn`, `info`, `debug`, `trace`. En producción, `info`; mientras investigás un problema, `debug`. Nunca loguees contraseñas, tokens ni datos personales completos.

Errores

- **Esperables** (datos inválidos, no encontrado, sin permiso): se responden con su status y un mensaje claro. Es lo que hace el manejador de errores de capas.
- **Inesperados** (un bug, la base caída): al cliente, un 500 genérico **sin** el stack ni el SQL; en los logs, todo el detalle. Un servicio como **Sentry** junta estos errores, los agrupa y avisa.
- **Fuera de un pedido** (`uncaughtException`, `unhandledRejection`): loguear y salir; el orquestador levanta otro proceso.

Apagado prolijo

Cada deploy reemplaza los procesos viejos por nuevos. Antes de apagar uno, la plataforma le manda `SIGTERM`. Si el proceso se corta en seco, los pedidos que estaba atendiendo fallan. Lo correcto: dejar de aceptar pedidos nuevos, terminar los que están en curso, cerrar las conexiones a la base y salir.

`src/demos/node/02-express/produccion/apagado.ts`

```
import express from 'express'
```

```ts
import { setTimeout as esperar } from 'node:timers/promises'

const app = express()
app.get('/salud', (_req, res) => {
  res.json({ ok: true })
})
app.get('/lento', async (_req, res) => {
  await esperar(300) // un pedido que tarda: no lo queremos cortar a la mitad
  res.json({ listo: true })
})

const servidor = app.listen(0, () => console.log('Servidor listo'))

// Docker, Kubernetes o el proveedor mandan SIGTERM antes de apagar o reemplazar la app.
// Apagado prolijo: dejar de aceptar pedidos, terminar los que están en curso y cerrar recursos.
process.on('SIGTERM', () => {
  console.log('SIGTERM: no acepto más pedidos, termino los que están en curso…')
  servidor.close(() => {
    console.log('Pedidos terminados. Cierro la base de datos y salgo.')
    process.exit(0)
  })
  // Si algo se cuelga, salir igual después de un tiempo
  setTimeout(() => process.exit(1), 10_000).unref()
})

// Para el ejemplo: un pedido lento y, mientras tanto, la señal de apagado
servidor.on('listening', async () => {
  const { port } = servidor.address() as { port: number }
  const pedido = fetch(`http://localhost:${port}/lento`).then((r) => r.json())
  await esperar(50)
  process.kill(process.pid, 'SIGTERM')
  console.log('El pedido lento igual respondió:', await pedido)
})
```

$ node apagado.ts · Node 24.22.1

```
Servidor listo
SIGTERM: no acepto más pedidos, termino los que están en curso…
El pedido lento igual respondió: { listo: true }
Pedidos terminados. Cierro la base de datos y salgo.
```

`servidor.close()` deja de aceptar conexiones nuevas y llama al callback cuando terminan las que estaban abiertas. El `setTimeout(..., 10_000).unref()` es un seguro: si algo se cuelga, sale igual (y `unref` evita que ese timer, por sí solo, mantenga vivo el proceso).

Health check

Un endpoint como `GET /salud` que responde 200 si la app está bien. Lo consultan el balanceador y el orquestador para saber si mandarle tráfico o reiniciarla. A veces hay dos: uno que dice "el proceso está vivo" y otro que además verifica que la base responde.

Deploy

```
# Dockerfile
FROM node:24-slim
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY src ./src
ENV NODE_ENV=production
USER node
EXPOSE 3000
CMD ["node", "src/servidor.ts"]
```

- **`npm ci --omit=dev`**: instalación exacta desde el lock y sin las dependencias de desarrollo.
- **`NODE_ENV=production`**: Express y muchas librerías se comportan distinto (más rápido, sin stacks en las respuestas de error).
- **`CMD ["node", ...]`** y no `npm start`: así la señal `SIGTERM` le llega directo a Node.
- **`USER node`**: no correr como root adentro del contenedor.

Dónde se publica una API de Node hoy: plataformas que toman el repo o el Dockerfile y se encargan del resto (**Railway**, **Render**, **Fly.io**), contenedores en la nube (**AWS ECS**, **Google Cloud Run**), o un VPS con Docker o **PM2** (un administrador de procesos que

reinicia la app si se cae y levanta una por núcleo). Las funciones *serverless* (**Vercel**, **AWS Lambda**) también corren Express, con matices (conexiones a la base, tiempos de arranque).

> 💡 **Lista para antes de salir**
>
> Configuración validada · logs estructurados · manejador de errores sin filtrar detalles · `SIGTERM` manejado · health check · `helmet`, CORS y rate limiting · pool de conexiones a la base con límite · timeouts en los `fetch` a otros servicios · `npm audit` limpio · tests en CI.

> 🎤 **En una entrevista**
>
> "¿Qué tenés en cuenta para poner una API de Node en producción?" → "Configuración por variables de entorno validadas al arrancar; logs estructurados con pino y errores a Sentry; un manejador de errores que no devuelve detalles internos; apagado prolijo con SIGTERM para no cortar pedidos en cada deploy; un health check; seguridad básica (helmet, CORS, rate limiting, validación); y varias instancias detrás de un balanceador, porque un proceso de Node usa un núcleo. La empaqueto en Docker con npm ci --omit=dev y NODE_ENV=production."

Parte 3 · Repaso

Todo junto para repasar antes de una entrevista.

Capítulo 16

Chuleta y preguntas de entrevista

Node y Express en una página, de Express 4 a 5 y preguntas frecuentes.

Todo en una página, para repasar antes de una entrevista o tener a mano en un proyecto.

Node

```
// Módulos
import { readFile } from 'node:fs/promises'
import { conIva } from './precios.ts'          // con extensión
export function f() {}                          // export con nombre
export default function g() {}                  // export por defecto
import.meta.dirname; import.meta.filename       // ex __dirname /
  __filename
if (import.meta.main) { /* solo si se ejecuta este archivo */ }

// Archivos y rutas
const texto = await readFile(path.join(import.meta.dirname,
  'datos.json'), 'utf8')
await writeFile(ruta, JSON.stringify(datos, null, 2))
await mkdir(carpeta, { recursive: true })
await rm(carpeta, { recursive: true, force: true })

// Streams
await pipeline(createReadStream(origen), createGzip(),
  createWriteStream(destino))
for await (const linea of createInterface({ input:
  createReadStream(ruta) })) {}
```

```
// Asincronía
const [a, b] = await Promise.all([pedirA(), pedirB()])        // en
  paralelo
const resultados = await Promise.allSettled(tareas)           // sin
  cortar por un error
await fetch(url, { signal: AbortSignal.timeout(5000) })       // con
  timeout
import { setTimeout as esperar } from 'node:timers/promises'  // await
  esperar(100)

// Proceso
process.argv.slice(2); parseArgs({ options: { ... } })   // argumentos
process.env.DB_HOST                                       // siempre
  string | undefined
process.exitCode = 1                                      // terminar
  con error
process.on('SIGTERM', apagar)

// Eventos
emisor.on('evento', fn); emisor.once('evento', fn);
  emisor.emit('evento', datos)
const [datos] = await once(emisor, 'evento')
```

```
node archivo.ts                      # ejecuta (TypeScript sin
  compilar desde Node 22.18)
node --watch archivo.ts              # reinicia al guardar
node --env-file=.env archivo.ts      # carga variables de entorno
node --test                          # corre los tests
npm ci --omit=dev                    # instalación de producción
```

Express

```
const app = express()
app.use(express.json())                          // req.body
app.use(cors({ origin: ['http://localhost:5173'] }))
app.use(helmet())

app.get('/ofertas', async (req, res) => {        // req.query, siempre
  strings
  res.json(await servicio.listar(Filtros.parse(req.query)))
})
app.get('/ofertas/:id', async (req, res) => {    // req.params.id
  const oferta = await servicio.obtener(Number(req.params.id))
  if (!oferta) return res.status(404).json({ error: 'No existe' })
  res.json(oferta)
```

```
})
app.post('/ofertas', autenticar, async (req, res) => {
  const oferta = await servicio.publicar(OfertaNueva.parse(req.body))
  res.status(201).location(`/ofertas/${oferta.id}`).json(oferta)
})
app.delete('/ofertas/:id', autenticar, soloAdmin, async (req, res) =>
  {
  await servicio.borrar(Number(req.params.id))
  res.status(204).end()
})

const router = express.Router()                    //
  app.use('/api/ofertas', router)

// Middleware
const autenticar: RequestHandler = (req, res, next) => {
  if (!req.get('authorization')) return res.status(401).json({ error:
    'Falta el token' })
  next()
}

// Al final: 404 y errores (4 parámetros)
app.use((req, res) => res.status(404).json({ error: 'No existe' }))
app.use((error, req, res, next) => {
  if (error instanceof ZodError) return res.status(400).json({
    detalles: error.issues })
  res.status(error.status ?? 500).json({ error: 'Error interno' })
}) satisfies ErrorRequestHandler)

if (import.meta.main) app.listen(3000)
```

De Express 4 a Express 5

Express 5 (estable desde 2024) es casi igual. Lo que cambió y vas a notar en un proyecto viejo:

	Express 4	Express 5
Errores en rutas async	Hay que atraparlos y llamar a `next(error)` (o usar `express-async-errors`)	Express los pasa solo al manejador de errores
Comodines en rutas	`app.get('*', ...)`, `'/archivos/*'`	Con nombre: `'/*todo'`, `'/archivos/*ruta'`; opcionales con llaves: `'/ofertas{/:id}'`

	Express 4	Express 5
`req.query`	Se podía reasignar	De solo lectura (un getter)
`req.body` sin parser	`{}`	`undefined`
Métodos viejos	`res.send(status)`, `res.json(obj, status)`, `req.param()`, `app.del()`	Eliminados: `res.status(201).json(obj)`
`res.redirect('back')`	Existía	`res.redirect(req.get('Referrer') ?? '/')`
Query anidada (`?a[b]=1`)	Se parseaba por defecto	No, salvo `app.set('query parser', 'extended')`
Node mínimo	0.10	18

Preguntas de entrevista

🎤 Node.js

- **¿Qué es Node?** Un entorno para ejecutar JavaScript fuera del navegador: el motor V8 más libuv, que le da I/O asincrónico (archivos, red, timers).
- **¿Es single-threaded?** Tu JavaScript corre en un solo hilo; el I/O lo hacen el sistema operativo y el thread pool de libuv. Para CPU pesado: worker threads.
- **¿Qué es el event loop?** El mecanismo que ejecuta los callbacks pendientes (timers, I/O, setImmediate) de a uno cuando el call stack está vacío. Entre cada uno se vacían las microtareas (promesas, nextTick).
- **¿Qué bloquea el event loop?** Código sincrónico largo: cálculos pesados, las funciones `Sync`, `JSON.parse` de algo enorme, regex catastróficas.
- **¿process.nextTick vs setImmediate vs setTimeout(0)?** nextTick corre antes de seguir con el loop (junto con las microtareas); setTimeout(0) en la fase de timers; setImmediate en la fase check, después del I/O.
- **¿CommonJS vs ES modules?** `require` / `module.exports`, sincrónico, vs `import` / `export`, el estándar, con top-level await y análisis estático.
- **¿Qué es un stream?** Datos procesados de a pedazos sin cargar todo en memoria; se conectan con `pipeline`, que maneja errores y backpressure.
- **¿Qué es un EventEmitter?** Un objeto que emite eventos con nombre a los que se suscriben oyentes; la base de streams, servidores y procesos.
- **¿dependencies vs devDependencies?** Las que se necesitan para correr vs las que solo se usan para desarrollar (tipos, tests, build).
- **¿Para qué es el package-lock.json?** Fija la versión exacta de todo el árbol de dependencias, para que todos (y el CI) instalen lo mismo.

- **¿Cómo usás todos los núcleos?** Varios procesos: cluster, PM2 o varias réplicas del contenedor detrás de un balanceador.

🎤 Express y APIs

- **¿Qué es un middleware?** Una función `(req, res, next)` en la cadena del pedido; responde o llama a `next()`. Los de error tienen 4 parámetros.
- **¿params, query y body?** Segmentos de la ruta, lo que va después del `?`, y el cuerpo (con `express.json()`). Todo viene del cliente: se valida.
- **¿PUT vs PATCH?** PUT reemplaza el recurso entero (idempotente); PATCH cambia una parte.
- **¿401 vs 403?** 401: no sé quién sos (falta o es inválido el token). 403: sé quién sos y no tenés permiso.
- **¿Qué es CORS?** Una protección del navegador: solo deja que una página lea respuestas de otro origen si el servidor lo autoriza con `Access-Control-Allow-Origin`.
- **¿Sesiones o JWT?** Sesión: el servidor guarda el estado, fácil de revocar. JWT: firmado, sin estado, difícil de revocar, por eso vida corta. En web, en cookie `httpOnly`.
- **¿Cómo se guardan las contraseñas?** Con un hash lento y con sal: bcrypt, argon2 o scrypt.
- **¿Cómo estructurás una API?** Por feature y en capas: rutas, servicio, repositorio, con las dependencias inyectadas.
- **¿Cómo manejás los errores?** Errores de negocio con su status, un único middleware que los traduce, un 500 genérico para lo inesperado y el detalle en los logs.
- **¿Cómo la testeás?** Unit tests de los servicios con mocks; integración con supertest o `listen(0)` y `fetch`.
- **¿Qué hacés en un deploy?** Variables validadas, logs estructurados, apagado prolijo con SIGTERM, health check, `npm ci --omit=dev`, `NODE_ENV=production`.